N054 Langfuse로 에이전트 문제 찾기_해설서

Chapter01

/강의정리 챕터 01. Langfuse로 에이전트 문제 찾기

오늘의 핵심 주제

LLMOps와 Langfuse를 활용한 AI 에이전트 관측 및 문제 분석

1. 오늘 수업의 목적

지금까지는 AI 에이전트를 만들고 정상적으로 작동하도록 하는 데 집중했다면, 이제부터는 실제 사용자가 서비스를 이용할 때 발생하는 문제를 찾아내고 개선하는 운영 단계로 넘어갑니다.

예를 들어 AI 친구 에이전트를 동료 5명에게 일주일간 사용하게 했더니 다음과 같은 문제가 발생했습니다.

사용자 불만	찾아야 할 원인
도쿄 날씨를 물었는데 엉뚱한 답변	잘못된 도구 선택, 검색 결과 오류
답변이 늦게 도착	LLM 응답 지연, 도구 실행 지연
이사 간다는 사실을 기억하지 못함	메모리 저장 또는 검색 실패
업데이트 후 답변이 느려짐	모델이나 코드 변경에 따른 성능 저하
별로인 답변이 있었음	답변 품질 문제
메시지에 비해 요금이 많이 나옴	불필요한 LLM 호출, 토큰 과다 사용

기존 Cloud Run 로그에는 `POST /api/chat 200` 같은 기록만 있습니다.

이것만으로는 어떤 단계에서 문제가 발생했는지 알기 어렵습니다.

따라서 오늘 수업의 목적은 Langfuse를 연결해 AI 에이전트의 내부 동작을 기록하고, 문제가 발생한 지점을 추적할 수 있는 시스템을 만드는 것입니다.

2. 중학생도 이해할 수 있는 핵심 개념

① LLMOps란 무엇인가?

일반적인 프로그램은 정해진 규칙에 따라 작동합니다.

예를 들어 계산기에 1+1을 입력하면 항상 2가 나옵니다.

하지만 AI는 다릅니다.

같은 질문을 해도 답변이 달라질 수 있고, 어떤 도구를 선택할지에 따라 결과도 달라집니다.

따라서 AI 서비스는 단순히 프로그램이 정상적으로 실행되는지만 확인해서는 부족합니다.

구분	일반적인 프로그램 운영	LLMOps
확인 대상	서버, 코드, 오류	서버 + AI의 판단과 결과
실행 결과	정해진 결과	확률적인 결과
문제 분석	오류 로그	질문, 모델, 도구, 응답 추적
성능 관리	실행 시간	응답 시간, 토큰, 비용
품질 관리	기능 정상 작동	답변의 정확성과 만족도

LLMOps는 AI 서비스를 안정적으로 운영하기 위한 관찰, 평가, 관리 체계입니다.

② Langfuse란 무엇인가?

Example - Tracing and Evaluation for the OpenAI-Agents SDK - Langfuse

Langfuse

AI 에이전트가 질문을 받고 답변하기까지 어떤 과정을 거쳤는지 기록하고 분석하는 도구입니다.

쉽게 말하면 AI 에이전트의 블랙박스 기록 장치입니다.

자동차가 사고 났을 때 블랙박스를 확인하면 사고 직전 어떤 일이 있었는지 알 수 있듯이, Langfuse를 사용하면 AI가 어떤 질문을 받고 어떤 도구를 사용했으며 어디에서 문제가 발생했는지 확인할 수 있습니다.

③ Trace, Span, Generation의 차이

오늘 수업에서 가장 중요한 개념입니다.

AI 친구에게 다음과 같이 질문했다고 가정하겠습니다.

"다음 주 도쿄 날씨를 알려줘."

이 질문에 답변하기까지 여러 단계가 진행됩니다.

AI 에이전트의 한 번의 대화 과정

Trace : 전체 대화 처리 과정

사용자 질문부터 최종 답변까지 전체 기록

Span 1. 사용자 질문 분석

질문의 의도와 필요한 작업 파악

Span 2. 날씨 도구 실행

도쿄 날씨 정보 요청

Span 3. LLM Generation

모델이 도구 결과를 바탕으로 답변 생성

Span 4. 최종 답변 전달

사용자에게 답변 반환

각 용어를 정리하면 다음과 같습니다.

개념	의미	비유
Trace	한 번의 전체 작업 기록	사건 전체 기록
Span	작업을 구성하는 개별 단계	사건의 개별 장면
Generation	LLM이 실제로 답변을 생성한 기록	AI가 생각하고 말한 과정

특히 Generation에는 모델 이름, 입력 및 출력 토큰, 소요 시간, 비용 등의 정보가 기록될 수 있습니다.

단, 모든 Span이 Generation은 아닙니다. 도구 실행이나 일반적인 처리 단계도 Span으로 기록할 수 있습니다.

④ 기록은 어떻게 Langfuse에 모일까?

AI 친구가 질문을 받으면 각 단계에서 관찰 데이터를 만듭니다.

이 데이터는 Langfuse SDK 등을 통해 수집되고, 전송 과정을 거쳐 Langfuse 화면에서 확인할 수 있습니다.

중요한 것은 모든 처리 과정을 기록하되 개인정보는 신중하게 관리하는 것입니다.

예를 들어 실제 대화 원문을 그대로 저장하는 대신 다음과 같이 기록할 수 있습니다.

- 사용자 ID: user_001
- 세션 ID: session_20261102
- 환경: production
- 버전: v1.2.0

- 실행 도구: weather_search
- 실행 시간: 1.2초
- 사용 토큰: 850개
- 결과 상태: 성공

개인정보 보호와 문제 분석이라는 두 가지 목적을 동시에 달성할 수 있습니다.

3. 동료들의 불만을 Langfuse로 해결하기

오늘 수업의 실전 핵심입니다.

발생한 문제	Langfuse에서 확인할 기능	확인할 내용
엉뚱한 답변	Trace 상세 화면	어떤 도구를 선택했고 결과가 무엇이었는가
응답이 느림	Trace 및 Span 시간	어느 단계에서 시간이 오래 걸렸는가
기억하지 못함	Trace 상세 화면	메모리 저장 및 검색 단계가 실행됐는가
업데이트 후 느려짐	버전별 Trace 비교	변경 전후 응답 시간과 토큰 사용량
답변 품질 저하	Score 및 사용자 피드백	좋아요, 별로예요, 신고가 발생한 요청
요금 증가	Generation 상세 정보	모델별 호출 횟수, 입력·출력 토큰, 비용

여기서 중요한 점은 Langfuse가 문제를 자동으로 해결하는 도구는 아니라는 것입니다.

문제가 발생한 위치와 원인을 찾을 수 있도록 근거를 제공하는 도구입니다.

4. 운영 단계에서 반드시 기록해야 할 정보

오늘 학습 목표 중 하나는 무엇을 기록해야 나중에 문제를 찾을 수 있는지 이해하는 것입니다.

기록 항목	필요한 이유
User ID	특정 사용자의 문제 추적
Session ID	한 사용자의 대화 흐름 확인
Environment	개발·테스트·운영 환경 구분
Version	배포 버전별 문제 비교
Trace ID	한 번의 요청 전체 추적
Span	단계별 실행 과정 확인
Model	어떤 AI 모델을 사용했는지 확인
Token	LLM 사용량과 비용 분석

기록 항목	필요한 이유
Latency	단계별 응답 속도 분석
Score	사용자 만족도 및 품질 평가

특히 User ID, Session ID, Environment, Version은 운영 단계에서 매우 중요합니다.

예를 들어 특정 동료가 업데이트 이후에만 문제를 경험했다면 사용자와 버전을 기준으로 트레이스를 필터링할 수 있습니다.

5. 사용자 피드백을 점수로 관리하기

사용자가 화면에서 누르는 좋아요, 별로예요, 신고 등의 반응도 중요한 운영 데이터입니다.

이를 Langfuse의 Score로 수집할 수 있습니다.

예를 들어 다음과 같이 관리할 수 있습니다.

사용자 반응	기록 점수 예시
좋아요	1
별로예요	0
신고	-1

이 점수는 예시이며, 실제로는 서비스에 맞는 평가 기준을 정해야 합니다.

이렇게 수집한 피드백을 활용하면 다음과 같은 일이 가능합니다.

1. 사용자 불만이 발생한 트레이스를 먼저 확인합니다.
2. 어떤 모델과 도구가 사용되었는지 분석합니다.
3. 문제가 반복되는 유형을 찾습니다.
4. 프롬프트나 도구 실행 로직을 개선합니다.
5. 개선 전후 결과를 비교합니다.

즉, 사용자의 주관적인 반응을 AI 서비스 개선에 활용할 수 있는 데이터로 전환하는 것입니다.

6. AI 개발자와 1인 AI 창업가가 알아야 하는 이유

AI 서비스를 만드는 것과 AI 서비스를 사업으로 운영하는 것은 다릅니다.

개발 단계에서는 기능이 작동하는지가 중요하지만, 사업 단계에서는 다음 네 가지가 중요합니다.

- 품질: 고객이 원하는 답변을 제공하는가?

- 속도: 고객이 기다리지 않아도 되는가?
- 비용: 매출 대비 AI 운영비가 적절한가?
- 신뢰: 문제가 발생했을 때 원인을 찾아 개선할 수 있는가?

특히 AI 컨설팅 사업이나 AI 에이전트 서비스를 운영하려면 고객에게 서비스가 잘 작동한다는 것을 설명할 수 있어야 합니다.

Langfuse는 이러한 운영 능력을 확보하는 데 도움이 됩니다.

7. 현재 진행 중인 프로젝트에 적용한다면?

기존에 진행해 온 AI 프로젝트 중 다음과 같이 활용할 수 있습니다.

프로젝트	Langfuse 적용 사례
AI 친구 에이전트	대화 품질, 기억 기능, 날씨 도구 오류 분석
뉴스레터 에이전트	뉴스 수집, 요약, 평가 단계별 성능 분석
MSM (My-Story-Maker)	원문 분석, 문체 분석, 웹소설 생성 단계별 품질 추적
AI 컨설팅 에이전트	고객 요구사항 분석, 자료 검색, 보고서 생성 과정 추적
Travel Creator	여행지 검색, 일정 구성, 추천 결과의 정확성 분석

특히 MSM에 적용하면 효과적입니다.

예를 들어 사용자가 일기를 입력하면 다음 과정이 진행됩니다.

MSM에 Langfuse를 적용한 예시

사용자 일기 입력

원문 분석

입력 글의 내용과 특징 분석

각 단계별 Span 기록

문체 분석

사용자의 고유한 표현과 말투 파악

각 단계별 Span 기록

웹소설 생성

원래 목소리를 유지하며 이야기 생성

각 단계별 Span 기록

결과 검증

원문과 생성 결과 비교

각 단계별 Span 기록

Trace 전체 기록 및 품질 평가

이렇게 하면 사용자가 "내 문체가 사라졌다"고 이야기했을 때 어느 단계에서 문제가 발생했는지 찾아갈 수 있습니다.

8. 오늘의 시작 질문 모범 답안

수업에서 제시한 주관식 퀴즈에 대한 답변입니다.

동료가 "도쿄 날씨를 물었는데 엉뚱한 답변이 나왔다"고 한 상황을 선택하겠습니다.

원인을 찾으려면 사용자의 질문, AI가 선택한 도구, 도구에 전달한 검색 조건, 실제로 반환된 날씨 정보, 최종 답변 생성에 사용된 모델과 프롬프트가 필요합니다.

현재 Cloud Run 로그에는 API 요청 성공 여부만 기록되어 있으므로 이러한 내부 처리 과정을 확인하기 어렵습니다. Langfuse를 도입해 한 번의 요청을 Trace로 기록하고, 질문 분석·도구 실행·LLM 답변 생성 과정을 각각 Span과 Generation으로 남기겠습니다.

또한 사용자 ID, 세션 ID, 배포 버전을 연결해 문제가 발생한 요청을 다시 찾을 수 있도록 하겠습니다. 대화 원문과 개인정보는 무분별하게 저장하지 않고 필요한 정보만 가려서 기록하겠습니다.

9. 오늘 반드시 기억해야 할 내용

핵심 개념 6가지

1. LLMOps: AI 서비스를 안정적으로 운영하고 개선하는 체계입니다.
2. Langfuse: AI 내부 실행 과정을 관찰하고 분석하는 도구입니다.
3. Trace: 한 번의 AI 요청 전체를 기록합니다.
4. Span: 전체 요청을 구성하는 개별 작업 단계입니다.
5. Generation: LLM이 실제로 생성한 답변과 관련 정보를 기록합니다.
6. Score: 사용자 피드백을 수치화하여 품질 개선에 활용합니다.

10. 한 문장으로 오늘 수업 정리

AI 에이전트를 만드는 것에서 끝나는 것이 아니라, Langfuse로 AI의 생각과 행동 과정을 기록하고 분석하여 답변 품질·속도·비용을 지속적으로 개선하는 것이 LLMOps의 핵심입니다.

11. 오늘 학습의 핵심 흐름

앞으로 배울 내용을 이해하기 위한 전체 구조

1단계. 문제 발생

사용자 불만

엉뚱한 답변, 느린 응답, 기억 실패, 비용 증가

2단계. 데이터 수집

Langfuse

Trace, Span, Generation 기록

3단계. 원인 분석

관측 및 비교

실행 단계, 시간, 토큰, 모델, 버전 확인

4단계. 개선

지속적 품질 개선

프롬프트, 도구, 메모리, 모델 및 운영 구조 개선

학습 방향에 대한 제안: 앞으로 AI 에이전트를 만들 때는 기능을 구현하기 전에 "나중에 이 기능이 잘못 작동하면 어디서 원인을 찾을 것인가?"를 먼저 생각하는 습관을 들이시면 좋겠습니다. 이것이 AI 서비스를 만드는 사람과 운영하는 사람의 차이를 만드는 중요한 역량입니다.

Chapter02

/강의정리 챕터 02. 로그로는 보이지 않는 것

오늘의 핵심 주제

서버 로그와 LLM 트레이스의 차이, LLMOps의 필요성, Langfuse의 작동 원리

1. 오늘 수업의 목적

지난 챕터에서는 Langfuse가 AI 에이전트의 내부 동작을 기록하고 분석하는 도구라는 것을 배웠습니다.

오늘은 한 단계 더 들어가서 왜 기존 서버 로그만으로는 AI 에이전트의 문제를 해결할 수 없는지, Langfuse는 어떤 구조로 기록을 만들고 관리하는지 이해하는 것이 목적입니다.

가장 중요한 차이는 다음과 같습니다.

- 서버 로그: 서버에 어떤 일이 발생했는가?
- LLM 트레이스: AI가 어떤 과정을 거쳐 그런 답변을 만들었는가?

예를 들어 AI 친구에게 도쿄 날씨를 물었는데 엉뚱한 답변이 나왔다고 가정하겠습니다.

서버 로그에는 다음과 같이 기록될 수 있습니다.

```
POST /api/chat 200
```

서버 입장에서는 요청이 정상적으로 처리되었기 때문에 성공입니다. 하지만 사용자는 잘못된 답변을 받았습니다.

이것이 LLM 서비스의 중요한 특징입니다.

기술적으로 성공한 요청이 사용자 관점에서는 실패한 서비스 경험일 수 있습니다.

2. 중학생도 이해할 수 있는 핵심 개념

① 서버 로그와 LLM 트레이스의 차이

택배 배송을 예로 들어 보겠습니다.

- 서버 로그는 택배가 접수되고 배송이 완료되었다는 기록입니다.
- LLM 트레이스는 택배가 어떤 경로로 이동했고, 어느 지점에서 문제가 생겼는지 보여 주는 상세 기록입니다.

두 기록 방식의 차이

서버 로그

결과 중심

POST /api/chat 200

요청 완료

내부 처리 과정은 알기 어려움

LLM 트레이스

과정 중심

질문 분석

도구 선택

도구 실행 실패

답변 생성

문제가 생긴 단계까지 추적

서버 로그가 불필요하다는 뜻은 아닙니다. 두 기록은 서로 보완하는 관계입니다.

서버 로그는 시스템의 건강 상태를 확인하고, LLM 트레이스는 AI의 내부 처리 과정을 분석합니다.

② LLM 서비스에 LLMOps가 필요한 네 가지 이유

일반 프로그램과 AI 프로그램은 작동 방식이 다릅니다.

첫째, 실행할 때마다 답변이 달라집니다.

일반 프로그램은 같은 입력에 같은 결과를 내도록 설계할 수 있습니다.

```
print(1 + 1)  # 항상 2
```

하지만 LLM은 확률적으로 다음 단어를 선택하기 때문에 같은 질문에도 다른 답변을 할 수 있습니다.

이것을 비결정론적(Non-deterministic)이라고 합니다.

따라서 문제가 발생한 바로 그 실행 기록을 보관해야 합니다. 나중에 같은 질문을 다시 보내는 것만으로는 당시의 문제를 재현하지 못할 수 있습니다.

둘째, 오류 메시지 없이 이상한 답을 합니다.

AI가 도쿄 날씨를 잘못 알려 주더라도 서버는 정상적으로 응답할 수 있습니다.

그래서 모델을 호출할 때 다음 정보를 기록해야 합니다.

기록 항목	확인 목적
전체 프롬프트와 컨텍스트	AI가 어떤 정보를 보고 답했는가
모델 설정	어떤 모델과 설정을 사용했는가
응답 시간	얼마나 오래 걸렸는가
입력·출력 토큰과 비용	얼마나 많은 자원을 사용했는가
실행 단계와 도구 결과	어떤 과정을 거쳐 답을 만들었는가

셋째, 비용을 사전에 예측하기 어렵습니다.

LLM API는 일반적으로 토큰을 기준으로 요금을 계산합니다.

토큰은 AI가 문장을 처리하기 위해 나누는 단위입니다. 글자 수와 정확히 일치하지 않으며 모델에 따라 차이가 있습니다.

이번 AI 친구 서비스는 메시지 하나에 모델을 세 번 호출합니다.

- 동료 5명
- 하루 20개 메시지씩
- 일주일 운영

계산하면 다음과 같습니다.

700개

일주일간 사용자 메시지

2,100회

모델 호출 횟수 (메시지당 3회)

5명 × 하루 20개 × 7일 = 700개 메시지

더 큰 문제는 대화가 길어지면 과거 대화와 기억까지 모델에 전달해야 하므로 입력 토큰이 증가한다는 점입니다.

따라서 모델별·단계별 토큰과 비용을 기록해야 불필요한 지출을 찾을 수 있습니다.

넷째, AI 답변은 단순히 통과와 실패로 평가하기 어렵습니다.

일반 프로그램은 정답이 명확한 경우가 많습니다.

```
assert add(2, 3) == 5
```

하지만 AI의 답변은 정답 하나로 판단하기 어렵습니다.

예를 들어 사용자가 "이사 준비 때문에 정신이 없어"라고 말했을 때 좋은 답변인지 판단하려면 다음을 고려해야 합니다.

- 사용자의 감정을 이해했는가?
- 이전 대화를 기억했는가?
- 적절한 말투를 사용했는가?
- 사실과 다른 내용을 만들지 않았는가?
- 안전한 답변을 했는가?

따라서 사람의 평가와 자동 평가를 함께 활용하는 품질 관리 체계가 필요합니다.

3. 서비스 운영 시 반드시 확인해야 할 다섯 가지 지표

지표	확인할 내용	실제 문제
응답 품질	사용자 평가와 만족도	별로인 답변
지연 시간	질문부터 답변까지 걸린 시간	답변이 느림
비용	토큰 사용량과 API 요금	요금 증가
오류율	API 실패와 시간 초과	시스템 장애
사용 패턴	도구와 기능의 이용 빈도	특정 기능의 반복적인 문제

이 다섯 가지는 AI 서비스를 사업으로 운영할 때 지속적으로 확인해야 하는 핵심 지표입니다.

4. LLMOps와 Langfuse의 관계

LLMOps는 AI 서비스를 개발하고 운영하면서 발생하는 문제를 관찰하고 개선하는 전체 활동입니다.

MLOps가 머신러닝 모델의 학습과 배포, 운영에 무게를 둔다면 LLMOps는 이미 학습된 LLM을 활용하는 과정, 즉 프롬프트·컨텍스트·모델 호출·비용·답변 품질 관리에 더 많은 관심을 둡니다.

LLMOps의 주요 구성 요소

트레이싱

AI가 어떤 과정을 거쳤는지 기록

비용 및 성능 모니터링

토큰, 비용, 응답 시간 분석

평가 (Evaluation)

사람과 AI가 답변 품질 평가

프롬프트 관리

프롬프트 버전 관리 및 비교

데이터셋과 실험

동일한 질문으로 개선 전후 비교

Langfuse는 이 중 여러 기능을 하나의 플랫폼에서 제공하는 오픈소스 LLMOps 도구입니다.

5. Langfuse의 데이터 구조: Trace와 Observation

오늘 강의에서 기술적으로 가장 중요한 부분입니다.

Langfuse는 크게 두 가지 구조로 데이터를 관리합니다.

- Trace: 요청 하나의 전체 처리 과정
- Observation: Trace 안에서 수행된 개별 작업

Observation에는 세 가지 기본 유형이 있습니다.

유형	의미	AI 친구 에이전트 사례
Span	일반적인 작업 단계	안전장치 검사, 날씨 API 호출
Generation	LLM 모델 호출	도구 선택, 답변 생성, 기억 정리
Event	특정 시점에 발생한 사건	기억 세 개를 불러옴

특히 Span과 Generation을 구분하는 것이 중요합니다.

Generation은 모델을 실제로 호출한 작업이므로 모델 이름, 입력·출력 토큰, 비용 등의 정보를 추가로 관리할 수 있습니다.

반면 Span은 모델 호출이 아닌 일반적인 처리 과정도 기록할 수 있습니다.

실제 에이전트에서의 기록 구조

다음은 AI 친구가 메시지 하나를 처리하는 과정을 트레이스로 기록한 예입니다.

Trace: run_turn

전체 요청

Span: check_safety

안전장치 검사

Generation: choose_tools

입력 820 / 출력 40 토큰

Span: run_tool

날씨 도구 실행 / 지원하지 않는 지역 반환

Generation: make_reply

입력 1,950 / 출력 120 토큰

Generation: update_memories

입력 1,100 / 출력 60 토큰

※ 토큰 수는 강의에서 제시한 설명용 예시입니다.

이 구조가 중요한 이유는 문제를 역으로 추적할 수 있기 때문입니다.

예를 들어 도쿄 날씨를 물었는데 AI가 엉뚱한 날씨를 알려 줬다면 다음을 확인합니다.

1. choose_tools: 날씨 도구를 제대로 선택했는가?
2. run_tool: 날씨 도구가 실제로 어떤 결과를 반환했는가?
3. make_reply: 모델이 도구 결과를 제대로 반영했는가?

이렇게 하면 단순히 답변이 잘못됐다는 사실을 넘어, 잘못된 답변이 처음 만들어진 단계를 찾을 수 있습니다.

6. 코드에 계측(Instrumentation)을 적용하는 방법

트레이스를 기록하려면 코드에 기록 지점을 만들어야 합니다.

이 작업을 계측(Instrumentation)이라고 합니다.

오늘 강의에서는 Python의 데코레이터(Decorator)를 이용합니다.

① 데코레이터란?

데코레이터는 기존 함수의 코드를 크게 변경하지 않고 추가 기능을 붙이는 방법입니다.

중학생에게 설명한다면 함수에 붙이는 자동 기록 스티커라고 생각하면 됩니다.

예를 들어 다음과 같습니다.

```
from langfuse import observe@observe()def check_safety(message):    ...@observe(as_typ
e="generation")def make_reply(message, tool_result):    ...
```

여기서 중요한 차이는 다음과 같습니다.

- `@observe()` : 일반적인 Span으로 기록
- `@observe(as_type="generation")` : LLM 호출을 Generation으로 기록

② 자동 중첩이란?

다음과 같이 여러 함수가 서로 호출한다고 가정하겠습니다.

```
@observe()def run_turn(message):    check_safety(message)    tool_result = run_tool(me
ssage)    return make_reply(message, tool_result)
```

가장 바깥쪽의 `run_turn()`이 실행되면 하나의 Trace가 만들어집니다.

그 안에서 실행되는 함수들은 자동으로 자식 Observation으로 연결됩니다.

이를 자동 중첩이라고 합니다.

즉, 모든 함수에 별도로 Trace ID를 전달하며 연결할 필요 없이 부모와 자식 관계를 기록할 수 있습니다.

다만 자동 중첩이 제대로 작동하려면 실행 컨텍스트가 올바르게 전파되어야 합니다.

7. Langfuse의 데이터 모델

Trace와 Observation 외에도 실제 운영에 필요한 데이터가 있습니다.

데이터	의미	활용 사례
User	서비스를 이용하는 사용자	특정 사용자의 문제 확인
Session	이어지는 대화의 묶음	한 대화 흐름 추적
Score	사람이나 프로그램의 평가	좋아요·별로예요 분석
Environment	실행 환경	staging과 production 구분
Version	배포 버전	업데이트 전후 비교
Tags	분류용 태그	도구 선택, 오류 유형 구분
Metadata	추가 속성 정보	도구 선택 확신도 등

특히 User, Session, Environment, Version은 운영 단계에서 매우 중요합니다.

예를 들어 업데이트 이후 답변이 느려졌다면 다음과 같이 분석할 수 있습니다.

- Version 1.1.0의 평균 응답 시간
- Version 1.2.0의 평균 응답 시간
- 동일한 환경에서 두 버전의 차이

이렇게 해야 업데이트가 실제로 성능에 영향을 주었는지 확인할 수 있습니다.

8. Langfuse는 어떻게 작동하는가?

Langfuse의 작동 구조는 크게 세 부분으로 나눌 수 있습니다.

① Langfuse SDK

서비스 코드에서 Trace와 Observation 생성

메모리 버퍼에 기록을 모음

② Background Thread

기록을 비동기 방식으로 전송

③ Langfuse Server

데이터 수신 및 저장·처리

④ Langfuse Web Dashboard

트레이스, 비용, 응답 시간, 평가 결과 확인

① SDK와 버퍼(Buffer)

SDK는 서비스 코드와 Langfuse를 연결해 주는 프로그램입니다.

코드가 실행되면서 생성된 기록은 즉시 서버로 전송되는 것이 아니라 우선 메모리의 버퍼에 쌓입니다.

강의에서 설명한 2026년 10월 기준 Python SDK 기본 설정은 다음과 같습니다.

- 기록 512개가 쌓이면 전송
- 또는 5초가 지나면 전송

비동기 전송이기 때문에 기록을 전송하는 작업이 사용자에게 답변을 보내는 작업을 직접 기다리게 하지 않습니다.

단, 프로세스가 강제로 종료되면 버퍼에 남아 있던 기록이 유실될 수 있습니다.

② flush()가 필요한 이유

예를 들어 평가용 메시지 20개를 처리하는 Python 스크립트를 실행했다고 가정하겠습니다.

스크립트는 정상적으로 실행됐지만 기록을 Langfuse에 보내기 전에 종료되면 일부 Trace가 보이지 않을 수 있습니다.

이때 사용하는 것이 `flush()` 입니다.

```
# 작업을 모두 마친 뒤 기록 전송langfuse.flush()
```

즉, 버퍼에 남아 있는 기록을 전송하도록 요청하는 것입니다.

오늘 퀴즈의 정답은 2번입니다.

정답 2번

SDK 버퍼에 남은 기록을 보내기 전에 프로세스가 끝났을 가능성이 가장 높습니다.

평가 스크립트처럼 실행하고 바로 종료하는 프로그램에서는 `flush()`를 챙기는 습관이 중요합니다.

③ OpenTelemetry(OTel)란?

OpenTelemetry는 프로그램의 트레이스, 지표, 로그를 수집하기 위한 공개 표준입니다.

쉽게 말하면 서로 다른 프로그램과 도구들이 동일한 방식으로 실행 기록을 주고받도록 만든 공통 언어입니다.

Langfuse Python SDK는 OpenTelemetry 기반으로 작동합니다.

이 덕분에 다른 라이브러리의 계측 데이터도 연동할 수 있습니다.

여기서 자동 계측(Auto-instrumentation)이라는 개념이 나옵니다.

예를 들어 LLM API 라이브러리의 호출을 자동으로 기록하도록 설정할 수 있습니다.

다만 자동 계측만으로는 AI 친구 에이전트의 `도구 선택`, `기억 정리` 같은 업무 단계를 구체적으로 알기 어렵습니다.

따라서 다음 두 가지를 함께 사용하는 것이 좋습니다.

- 자동 계측: 라이브러리의 모델 호출 등을 기록
- 수동 계측: 우리 서비스만의 업무 단계와 도구 실행 과정을 기록

9. Langfuse Cloud와 Self-hosting 비교

Langfuse를 사용하는 방법은 두 가지입니다.

Example - Tracing and Evaluation for the OpenAI-Agents SDK - Langfuse

Langfuse Cloud

Langfuse 회사가 서버를 운영합니다.

- 별도 설치 불필요
- 빠른 시작
- 서버 관리 부담 적음

Docker Composeで３層アーキテクチャ（JS + Python + MySQL）を構築する #JavaScript - Qiita

Self-hosting

내 서버에 직접 설치하고 운영합니다.

- 데이터 관리 통제
- 인프라 운영 필요
- 유지보수 부담 증가

비교	Cloud	Self-hosting
서버 운영	외부 서비스	직접 운영
설치	간편	복잡
관리 부담	낮음	높음
데이터 통제	제공사 정책에 따름	직접 관리
적합한 경우	학습, 초기 개발	보안 규제, 특수한 운영 환경

Self-hosting은 단순히 프로그램 하나를 설치하는 것이 아닙니다.

Web과 Worker 컨테이너 외에도 Postgres, ClickHouse, Redis, 오브젝트 스토리지 등을 운영해야 합니다.

이번 강의에서는 설치와 운영 부담이 적은 Langfuse Cloud를 사용합니다.

강의 자료의 무료 플랜 한도는 2026년 10월 기준이므로 실제 프로젝트에 도입할 때는 최신 요금 정책을 다시 확인해야 합니다.

10. 현재 진행 중인 프로젝트에 적용하기

오늘 배운 내용을 실제 프로젝트에 적용한다면 다음과 같습니다.

프로젝트	기록해야 할 단계	분석 목적
AI 친구 에이전트	질문 분석 → 도구 선택 → 실행 → 답변 → 기억	답변 오류와 기억 실패
뉴스레터 에이전트	뉴스 수집 → 평가 → 요약 → 발송	뉴스 품질과 호출 비용
MSM	원문 분석 → 문체 분석 → 이야기 생성 → 검증	원래 사용자의 문체 보존
AI 컨설팅 에이전트	요구 분석 → 자료 검색 → 진단 → 보고서 작성	분석 정확도와 보고서 품질
Travel Creator	여행지 검색 → 일정 구성 → 추천 → 검증	여행 정보 정확성과 응답 속도

특히 MSM은 원문을 보존하는 것이 핵심이므로 단계별 입력과 출력 기록이 중요합니다.

예를 들어 최종 웹소설이 사용자의 원래 문체와 너무 달라졌다면, 원문 분석 단계의 문제인지 문체 분석 단계인지 이야기 생성 단계인지 구분할 수 있어야 합니다.

이런 구조가 만들어지면 단순한 AI 글쓰기 도구가 아니라 품질을 지속적으로 관리하는 AI 서비스로 발전시킬 수 있습니다.

11. AI 개발자와 1인 창업가가 알아야 할 이유

오늘 수업에서 가장 중요하게 생각할 부분입니다.

AI 서비스를 만드는 사람은 크게 두 가지 능력이 필요합니다.

첫째, AI 서비스를 만드는 능력

- 프롬프트 설계
- 모델 연결
- 도구 연결
- 에이전트 워크플로 구성
- 데이터와 메모리 관리

둘째, AI 서비스를 운영하고 개선하는 능력

- 트레이스 분석
- 비용과 토큰 관리
- 응답 속도 관리
- 사용자 만족도 평가
- 버전별 성능 비교

첫 번째 능력만 있으면 AI 프로그램을 만들 수 있습니다.

하지만 두 번째 능력까지 있어야 고객에게 안정적인 서비스를 제공하고 사업으로 운영할 수 있습니다.

특히 비개발자 창업가에게는 Langfuse 같은 도구가 중요합니다. 코드를 직접 모두 이해하지 못하더라도 관측 데이터를 통해 개발자나 코딩 에이전트에게 문제를 구체적으로 설명하고 개선을 요청할 수 있기 때문입니다.

12. 오늘 반드시 기억해야 할 내용

핵심 개념 10가지

1. 서버 로그: 요청과 서버의 실행 결과를 기록합니다.
2. LLM 트레이스: AI의 내부 처리 과정을 기록합니다.
3. LLMOps: AI 서비스를 관찰하고 평가하며 개선하는 운영 체계입니다.
4. Trace: 요청 하나의 전체 처리 과정입니다.
5. Observation: Trace 안에 있는 개별 작업 기록입니다.
6. Span: 일반적인 작업 단계를 기록합니다.
7. Generation: LLM 모델 호출과 토큰·비용을 기록합니다.

8. Instrumentation: 코드에 관찰 기록 지점을 넣는 작업입니다.
9. flush(): 버퍼에 남은 기록을 전송하도록 요청합니다.
10. OpenTelemetry: 여러 시스템의 관측 데이터를 연결하는 공개 표준입니다.

오늘 배운 기술들의 관계

기술	쉽게 말하면
Python Decorator	함수에 기록 기능을 붙이는 방법
Langfuse SDK	관찰 기록을 만드는 프로그램
Buffer	기록을 잠시 모아 두는 공간
Background Thread	기록을 백그라운드에서 전송하는 작업
OpenTelemetry	관측 데이터의 공통 규칙
Langfuse Dashboard	수집된 기록을 확인하는 화면

13. 다음 강의를 위한 생각거리

다음 챕터부터 실제 계측을 진행하기 전에 다음 질문을 생각해 보시면 좋겠습니다.

AI 친구 에이전트의 한 메시지를 처리하는 다섯 단계 중 어디에 반드시 기록을 남겨야 할까요?

제 생각에는 다음 원칙이 중요합니다.

- 모델을 호출하는 곳은 Generation으로 기록합니다.
- 일반적인 기능 실행은 Span으로 기록합니다.
- 도구의 입력과 반환 결과를 기록합니다.
- 각 단계의 실행 시간과 오류 여부를 기록합니다.
- 개인정보와 민감한 대화 내용은 별도의 보호 정책을 적용합니다.

특히 실제 서비스에서는 모든 대화 원문을 무조건 기록하는 것이 아니라, 분석에 필요한 정보와 개인정보 보호 사이의 균형을 설계해야 합니다.

14. 한 문장으로 오늘 수업 정리

서버 로그가 AI 서비스의 겉으로 드러난 실행 결과를 보여 준다면, Langfuse 트레이스는 AI가 어떤 입력과 판단, 도구 실행을 거쳐 결과를 만들었는지 보여 주며, 이것이 LLMOps의 출발점입니다.

오늘의 학습 핵심 흐름

기록 → 추적 → 분석 → 개선

AI 서비스를 개발하는 데서 끝나지 않고, 실제 운영 과정에서 발생하는 문제를 데이터로 찾아내고 개선하는 역량을 익히는 과정입니다.

학습 방향에 대한 제안: 앞으로 만들 ASP의 AI 컨설팅 서비스에도 처음부터 Langfuse 계측 구조를 고려하시길 권합니다. 고객에게 제공하는 진단 결과가 어떤 자료와 어떤 AI 처리 단계를 통해 만들어졌는지 추적할 수 있다면, 컨설팅 결과의 신뢰성과 재현성을 높이는 데 도움이 됩니다.

Chapter03

/강의정리 챕터 03. Langfuse 실습 환경 구축과 연결

오늘의 핵심 주제

실습 환경 점검, Langfuse Cloud 프로젝트 생성, API 키 설정

1. 오늘 수업의 목적

챕터 01에서는 Langfuse가 무엇인지, 챕터 02에서는 왜 필요한지와 내부 작동 원리를 배웠습니다.

오늘은 실제로 Langfuse를 사용하기 위한 준비 단계입니다.

핵심은 세 가지입니다.

1. 기존 AI 친구 에이전트 서비스가 정상적으로 실행되는지 확인합니다.
2. Langfuse Cloud에 프로젝트를 만들고 API 키를 발급받습니다.
3. 기존 에이전트 서비스와 Langfuse를 연결할 준비를 합니다.

오늘 수업은 본격적인 계측 코드를 작성하기 전에 AI 에이전트와 관측 도구가 서로 연결될 수 있는 환경을 구축하는 과정입니다.

2. 실습 환경의 전체 구조 이해하기

이번 실습은 두 가지 환경에서 진행됩니다.

구분	사용 환경	실습 내용
섹션 4~6	내 컴퓨터(Local)	계측 코드 적용 및 트레이스 확인

구분	사용 환경	실습 내용
섹션 7 이후	Staging	실제 배포 환경에서 트레이스 확인
Production	실제 서비스 운영 환경	운영 환경과 배포 버전 구분

쉽게 말하면 다음과 같습니다.

- Local: 내 작업실에서 실험하는 공간
- Staging: 실제 서비스를 배포하기 전에 최종 점검하는 공간
- Production: 고객이 실제로 이용하는 운영 공간

AI 친구 에이전트의 실습 환경

Local (내 컴퓨터)

코드 수정 및 계측 실험

Staging (Cloud Run)

배포 후 동작 및 관측 확인

Production (Cloud Run)

실제 서비스 운영

Langfuse Cloud

Local, Staging, Production의 트레이스를 수집하고 분석

여기서 중요한 점은 Local과 Staging, Production이 하나의 Langfuse 프로젝트에 연결될 수 있다는 것입니다.

환경을 구분하는 것은 별도의 프로젝트를 만드는 대신 `environment`라는 속성을 활용합니다.

이렇게 하면 하나의 화면에서 환경별 성능을 비교할 수 있습니다.

3. 실습 시작 전에 확인해야 할 다섯 가지

오늘 수업에서는 기존에 구축한 환경을 다시 확인합니다.

확인 항목	의미	필요한 이유
저장소와 가상환경	Python 프로젝트 실행 준비	계측 코드 적용
실제 LLM API 키	실제 모델 호출 가능 여부	토큰과 비용 확인
Cloud Run 서비스	Staging과 Production 존재	배포 환경 실습
GitHub Actions	자동 배포 워크플로	변경 코드 배포
Supabase 연결	사용자 인증과 식별	사용자별 트레이스 구분

특히 LLM API 키가 중요합니다.

Fake LLM과 실제 LLM의 차이

구분	Fake LLM	실제 LLM
응답 생성	가짜 응답	실제 모델 응답
트레이스 구조	확인 가능	확인 가능
토큰 사용량	0 또는 실제 사용량 없음	실제 토큰 기록
비용 분석	불가능	가능
실전 운영 재현	제한적	가능

Fake LLM은 실제 AI 모델을 호출하지 않고 가상의 결과를 반환하는 테스트 방식입니다.

따라서 계측 구조를 테스트할 때는 유용하지만 실제 LLM 호출 비용이 어떻게 발생하는지 확인할 수는 없습니다.

오늘 수업의 핵심은 실제 LLM 호출과 Langfuse 기록을 연결할 준비를 하는 것입니다.

4. 명령어로 시작 상태 확인하기

이번 수업에서는 기존에 만든 프로젝트를 확인하는 명령어 세 가지를 배웁니다.

① Python 프로젝트가 정상적으로 불러와지는지 확인

```
python -c "import app.main" && echo ok
```

이 명령은 Python이 `app.main` 모듈을 정상적으로 읽을 수 있는지 확인합니다.

- 성공: `ok` 출력
- 실패: 모듈 경로나 설치된 패키지 등을 점검

여기서 중요한 것은 서버를 실제로 실행하는 것이 아니라 Python 코드가 정상적으로 import되는지 확인한다는 점입니다.

② Cloud Run 서비스 확인

```
gcloud run services list --region us-central1
```

Google Cloud Run에 등록된 서비스를 확인하는 명령입니다.

여기서 Staging과 Production 서비스가 모두 있는지 확인합니다.

③ 자동 배포 설정 확인

```
grep -n "docker build" .github/workflows/deploy.yml
```

GitHub Actions의 배포 파일에서 Docker 이미지 빌드 명령이 있는지 확인합니다.

즉, 코드를 수정한 다음 새로운 실행 이미지를 만들고 배포할 수 있는 구조인지 확인하는 것입니다.

5. Langfuse Cloud 프로젝트 만들기

Deploy Langfuse on Northflank — Stack Templates — Northflank

Langfuse Cloud

AI 서비스의 트레이스와 비용을 수집하고 분석하는 클라우드 서비스입니다.

미국 리전 Langfuse 접속

이번 강의에서는 직접 서버를 설치하지 않고 Langfuse Cloud를 이용합니다.

① 왜 미국 리전을 선택할까?

이번 AI 친구 에이전트는 Google Cloud Run의 미국 리전인 `us-central1`에서 실행됩니다.

따라서 Langfuse도 미국 리전을 선택하면 데이터를 전송하는 거리를 줄일 수 있습니다.

구분	미국	유럽
접속 주소	`https://us.cloud.langfuse.com`	`https://cloud.langfuse.com`
이번 실습	권장	사용하지 않음

여기서 반드시 기억할 점이 있습니다.

Langfuse 가입 시 선택한 리전과 SDK가 데이터를 보내는 접속 주소가 일치해야 합니다.

주소가 잘못되면 API 키가 정확하더라도 트레이스가 정상적으로 들어오지 않을 수 있습니다.

② 프로젝트 생성

Langfuse에 가입한 뒤 다음 순서로 진행합니다.

1. Organization 생성
2. Project 생성
3. 프로젝트 이름 지정: `friend-agent`

4. Settings 메뉴 이동
5. API Keys 선택
6. 새로운 API 키 발급

Staging과 Production은 별도 프로젝트로 나누지 않고 하나의 프로젝트 안에서 환경 속성으로 구분합니다.

이렇게 하면 프로젝트 관리가 단순해지고 환경별 비교도 편리해집니다.

6. API Key의 구조와 역할

오늘 실습에서 매우 중요한 부분입니다.

Langfuse API 키는 두 가지로 구성됩니다.

키 종류	시작 문자열	역할
Public Key	`pk-lf-`	프로젝트 식별
Secret Key	`sk-lf-`	인증 및 기록 전송 권한

쉽게 비유하면 다음과 같습니다.

- Public Key: 어느 건물에 데이터를 보내는지 알려 주는 주소표
- Secret Key: 실제로 건물에 들어갈 수 있는 출입증

Public Key는 혼자서는 기록을 전송할 수 없습니다. Secret Key가 있어야 인증이 가능합니다.

특히 Secret Key는 외부에 노출되면 안 됩니다.

① .env 파일에 저장하기

저장소 루트에 있는 `.env` 파일에 다음 세 가지 환경변수를 추가합니다.

```
LANGFUSE_PUBLIC_KEY=pk-lf-...
LANGFUSE_SECRET_KEY=sk-lf-...
LANGFUSE_BASE_URL=https://us.cloud.langfuse.com
```

실제 키는 자신의 Langfuse 프로젝트에서 발급받은 값을 넣어야 합니다.

보안 주의

Secret Key는 다음과 같이 관리해야 합니다.

- 채팅창이나 공개된 명령어에 붙여 넣지 않습니다.

- GitHub 저장소에 커밋하지 않습니다.
- `.env`는 Git 추적 대상에서 제외합니다.
- 배포 환경에서는 Secret Manager에 저장합니다.

② .env.example도 수정하기

`.env.example`은 다른 개발자가 프로젝트를 내려받았을 때 어떤 환경변수가 필요한지 알려주는 안내서입니다.

따라서 실제 키는 넣지 않고 변수 이름만 기록합니다.

```
LANGFUSE_PUBLIC_KEY=
LANGFUSE_SECRET_KEY=
LANGFUSE_BASE_URL=
```

이렇게 하면 새로운 개발 환경을 만들 때 필요한 설정을 빠뜨릴 가능성이 줄어듭니다.

7. Local과 Cloud Run의 환경변수 관리 차이

이 부분은 앞서 배운 Cloud Run과 GitHub Actions 수업과 연결됩니다.

구분	Local	Cloud Run
실행 환경	내 컴퓨터	Google Cloud
환경변수 설정	`.env`	Cloud Run 설정
Secret 관리	로컬 파일 보호	Secret Manager
Langfuse 연결	SDK가 `.env` 읽음	배포 환경에서 키 전달

즉, Local에서는 `.env` 파일을 활용하고 Cloud Run에서는 Secret Manager를 이용해 비밀키를 안전하게 관리하는 것이 기본적인 구조입니다.

여기서 기억할 점은 코드와 비밀번호를 분리하는 것입니다.

코드는 GitHub에 올라가도 되지만 비밀키는 GitHub에 올라가면 안 됩니다.

8. 오늘의 객관식 퀴즈

문제

Langfuse 키를 넣고 계측까지 했는데 트레이스가 하나도 들어오지 않습니다. 오류 메시지도 없습니다. 가장 먼저 확인할 것은 무엇일까요?

1. Cloud Run 인스턴스 수

2. `LANGFUSE_BASE_URL`이 가입할 때 고른 리전의 주소와 같은지

3. LLM API 키의 잔액

4. Supabase 익명 로그인 설정

정답 확인

9. AI 개발자와 1인 창업가 관점에서 알아야 할 이유

오늘 수업은 단순히 API 키를 발급받는 과정이 아닙니다.

AI 서비스를 사업으로 운영하려면 다음 세 가지를 구분할 수 있어야 합니다.

1. 개발 환경: 코드를 만들고 실험하는 곳
2. 운영 환경: 고객이 실제로 서비스를 이용하는 곳
3. 관측 환경: 서비스가 제대로 작동하는지 확인하는 곳

이 세 가지가 연결되어야 서비스의 문제를 발견하고 개선할 수 있습니다.

특히 ASP의 AI 컨설팅 서비스나 MSM처럼 고객의 데이터와 결과물을 다루는 서비스에서는 보안과 운영 관리가 중요합니다.

Secret Key 관리, 환경별 분리, 사용자별 추적은 처음부터 설계하는 것이 좋습니다.

10. 오늘 반드시 기억해야 할 내용

핵심 개념 8가지

1. Local은 개발과 실험을 위한 환경입니다.
2. Staging은 배포 전 검증 환경입니다.
3. Production은 실제 운영 환경입니다.
4. Fake LLM은 트레이스 구조 확인에 유용하지만 실제 비용 분석은 불가능합니다.
5. Langfuse Cloud 프로젝트는 하나로 만들고 environment로 실행 환경을 구분할 수 있습니다.
6. Public Key는 프로젝트 식별에 사용하고 Secret Key는 인증에 사용합니다.
7. `.env`는 로컬 환경에서 사용하고 배포 환경의 비밀키는 Secret Manager로 관리합니다.
8. `LANGFUSE_BASE_URL`은 가입한 Langfuse 리전의 주소와 일치해야 합니다.

11. 현재 진행 중인 프로젝트에 적용한다면?

오늘 배운 내용은 앞으로 진행할 프로젝트의 공통적인 개발 기반으로 활용할 수 있습니다.

프로젝트	적용할 내용
AI 친구 에이전트	Local·Staging·Production별 Langfuse 연결
MSM	개발 환경과 운영 환경의 트레이스 분리
뉴스레터 에이전트	모델별 토큰 및 비용 추적
ASP 컨설팅 에이전트	고객별 접근 권한과 비밀키 관리
Travel Creator	여행지 검색과 일정 생성 과정 관측

특히 앞으로 여러 AI 서비스를 운영하게 된다면 Langfuse 프로젝트를 서비스별로 관리하고, 각 프로젝트 안에서 환경을 구분하는 운영 원칙을 정해 두는 것이 좋겠습니다.

12. 한 문장으로 오늘 수업 정리

Langfuse를 실제 AI 서비스에 연결하려면 Local·Staging·Production 환경을 점검하고, 올바른 리전의 프로젝트와 API 키를 생성한 뒤, 환경변수와 Secret Manager를 통해 안전하게 연결해야 합니다.

다음 단계: 다음 챕터에서는 실제 코드에 Langfuse SDK를 연결하고, AI 에이전트의 어느 함수에 어떤 관찰 단위를 적용할지 구체적으로 설계하는 과정으로 넘어가게 됩니다.

Chapter04

/강의정리 챕터 04. 엉뚱한 날씨 답 — 에이전트 계측 구조 설계

오늘의 핵심 주제

AI 에이전트의 한 번의 대화를 단계별로 기록하고, 문제가 발생한 원인을 추적할 수 있도록 설계하기

1. 오늘 수업의 목적

동료가 AI 친구 에이전트에게 "어제 저녁 도쿄 날씨가 어땠어?"라고 물었는데 엉뚱한 답을 받았습니다.

그런데 서버 로그에는 다음과 같은 기록만 남아 있습니다.

```
POST /api/chat 200
get_weather 선택
POST /api/chat 200
```

이 기록만으로는 무엇이 잘못됐는지 알 수 없습니다.

- 사용자가 실제로 어떤 질문을 했는가?
- AI가 왜 날씨 도구를 선택했는가?
- 날씨 도구는 어떤 결과를 반환했는가?
- AI는 그 결과를 보고 어떻게 답변했는가?

오늘 수업의 목적은 이런 문제를 해결하기 위해 한 번의 대화를 하나의 트레이스로 묶고, 내부 처리 과정을 단계별로 기록하는 구조를 설계하는 것입니다.

2. 서버 로그와 트레이스의 차이

중학생도 이해할 수 있도록 병원에 비유해 보겠습니다.

- 서버 로그는 병원에 누가 몇 시에 들어왔고 나갔는지를 기록하는 출입 기록입니다.
- 트레이스는 환자가 접수하고, 검사받고, 진단받고, 처방받는 과정을 단계별로 기록한 진료 기록입니다.

서버 로그만 있으면 병원에 방문했다는 사실은 알지만, 왜 잘못된 진단을 내렸는지 알기 어렵습니다.

AI 서비스도 마찬가지입니다.

구분	서버 로그	Langfuse 트레이스
요청 발생	확인 가능	확인 가능
사용자 메시지	제한적	기록 가능
모델 호출	제한적	단계별 확인
도구 실행 결과	별도 기록 필요	입력·출력 기록 가능
모델 답변	별도 기록 필요	기록 가능
토큰과 비용	확인 어려움	Generation으로 추적
문제 원인 분석	제한적	단계별 분석 가능

핵심은 요청이 성공했는지가 아니라, 요청 안에서 어떤 일이 일어났는지를 확인하는 것입니다.

3. 한 번의 대화를 다섯 단계로 나누기

오늘 수업에서 가장 중요한 내용입니다.

AI 친구 에이전트가 사용자 메시지를 받으면 다음 다섯 단계를 거칩니다.

AI 에이전트 한 턴의 처리 과정

1. 안전장치 (Safety)

   사용자의 메시지가 위기 표현이나 위험한 요청에 해당하는지 확인합니다.

2. 도구 선택 (Choose Tools)

   날씨 검색, 공휴일 조회 등 어떤 도구를 사용할지 결정합니다.

3. 도구 실행 (Tool Execution)

   선택된 도구를 실제로 실행하고 결과를 가져옵니다.

4. 답 만들기 (Reply)

   도구 실행 결과 등을 바탕으로 사용자에게 전달할 답변을 만듭니다.

5. 기억 정리 (Memory Update)

   대화에서 기억할 내용을 추가하거나 수정하거나 삭제합니다.

추가 단계: 대화 요약 (Memory Summary)

요약되지 않은 메시지가 40개를 넘으면 별도로 요약 작업을 수행합니다.

왜 이렇게 나눌까요?

문제가 발생했을 때 어느 부분을 고쳐야 하는지 알 수 있기 때문입니다.

예를 들어 도구를 잘못 선택했다면 도구 선택 프롬프트나 규칙을 수정하고, 도구가 잘못된 데이터를 반환했다면 도구 실행 코드를 수정해야 합니다.

즉, 서로 다른 원인으로 문제가 발생할 수 있는 단계는 별도로 기록해야 합니다.

트레이스는 무엇을 기준으로 만들까요?

`run_turn` 함수가 한 번 실행되는 것이 사용자 메시지 하나를 처리하는 과정입니다.

따라서 메시지 하나를 하나의 트레이스로 만듭니다.

사용자가 처음 접속했을 때 실행되는 `greet`는 `/api/greeting`이라는 별도 요청이므로 별도의 트레이스로 관리합니다.

4. Span과 Generation 구분하기

다섯 단계를 나눈 다음에는 각 단계가 어떤 종류의 관찰 단위인지 정해야 합니다.

가장 중요한 기준은 **그 단계에서 실제 LLM을 호출하는가?**입니다.

단계	LLM 호출 여부	Langfuse 종류
안전장치	호출하지 않음	Span
도구 선택	실제 LLM 사용 시 호출	Generation
도구 실행	호출하지 않음	Span
답 만들기	호출함	Generation
기억 정리	실제 LLM 사용 시 호출	Generation
대화 요약	요약할 때 호출	Generation

쉽게 이해하기

- Span: AI 모델을 호출하지 않고 수행하는 일반적인 작업
- Generation: 실제 AI 모델을 호출하는 작업

예를 들어 날씨 API를 호출하는 것은 도구 실행이므로 Span입니다.

반면 날씨 정보를 바탕으로 답변을 생성하는 것은 AI 모델을 호출하므로 Generation입니다.

이 구분이 중요한 이유는 Generation에 모델 이름과 토큰 사용량을 기록하면 모델 호출별 비용을 계산할 수 있기 때문입니다.

도구 선택도 항상 Generation인 것은 아닙니다. Fake LLM을 사용하거나 규칙 기반으로 선택하면 실제 모델 호출이 없으므로 토큰과 비용은 발생하지 않습니다.

따라서 `decided_by`라는 속성으로 모델이 선택했는지, 규칙이 선택했는지도 기록합니다.

5. 단계별로 무엇을 기록할 것인가?

단순히 단계가 실행됐다는 사실만 기록해서는 부족합니다.

각 단계의 입력과 출력이 있어야 원인을 찾을 수 있습니다.

단계	기록할 내용	분석 목적
한 턴 전체	사용자 메시지, 최종 답변, 메시지 ID	전체 흐름 확인
안전장치	위기 표현 감지 여부	안전장치 작동 확인
도구 선택	모델 입력·출력, 선택 도구, 인자, 확신도, 결정 주체	도구 선택 오류 확인
도구 실행	도구 인자, 실행 결과, 성공 여부	도구 오류 확인
답 만들기	모델 입력·출력, 토큰, 첫 글자 출력 시각	답변 품질·속도·비용 확인

단계	기록할 내용	분석 목적
기억 정리	추가·수정·삭제한 기억	기억 오류 및 비용 확인
대화 요약	요약 대상 메시지 수, 새 요약	요약 결과 및 비용 확인

특히 다음 세 가지는 꼭 기억해야 합니다.

1. 도구 선택: 어떤 도구를 왜 골랐는지 확인합니다.
2. 도구 실행: 실제 도구가 어떤 결과를 반환했는지 확인합니다.
3. 답 만들기: AI가 그 결과를 어떻게 활용했는지 확인합니다.

도쿄 날씨가 엉뚱하게 나온 경우 이 세 가지를 나란히 비교하면 문제 원인을 좁힐 수 있습니다.

답변 속도도 단계별로 측정해야 합니다.

AI 답변은 스트리밍 방식으로 전달될 수 있습니다.

사용자가 답변을 기다리는 시간은 모델이 모든 답변을 완성한 시점이 아니라 첫 글자가 도착하는 시점부터 체감되기 시작합니다.

따라서 `reply`에는 첫 글자가 나온 시각도 기록합니다.

또한 기억 정리는 사용자가 답을 받은 뒤에 실행될 수 있습니다. 사용자가 답변을 받았더라도 서버 작업과 모델 호출이 계속될 수 있으므로, 실제 비용은 답변 전달 이후에도 발생할 수 있습니다.

6. Langfuse에서 사용할 관찰 단위 이름 정하기

관찰 단위에는 일관된 이름을 붙여야 합니다.

구분	관찰 단위 이름	종류
한 턴 전체	`chat-turn`	Trace
안전장치	`safety`	Span
도구 선택	`choose-tools`	Generation
도구 실행	`tool:<도구 이름>`	Span
답 만들기	`reply`	Generation
기억 정리	`memory-update`	Generation
대화 요약	`memory-summary`	Generation
먼저 말 걸기	`greeting`	Trace

왜 함수 이름 대신 별도의 이름을 사용할까요?

코드의 함수 이름은 개발하면서 변경될 수 있지만, Langfuse에서 사용하는 이름은 분석 기준이기 때문입니다.

예를 들어 `reply`라는 이름을 계속 사용하면 여러 트레이스에서 답변 생성 단계만 모아 속도와 비용을 비교할 수 있습니다.

이름을 통일하는 것은 단순한 코드 작성 규칙이 아니라 운영 데이터를 비교할 수 있게 만드는 기준입니다.

7. 트레이스 속성과 개인정보 관리

트레이스에는 단계별 기록 외에도 사용자를 구분하거나 실행 환경을 구분하는 속성을 붙일 수 있습니다.

대표적인 속성은 다음과 같습니다.

속성	의미	활용
User	사용자 식별값	사용자별 문제 분석
Session	대화 묶음	대화 흐름 확인
Score	사용자 평가	답변 품질 분석
Environment	local, staging, production	환경별 비교
Version	실행 코드 버전	업데이트 전후 비교
Tags	분류용 꼬리표	문제 유형별 검색
Metadata	추가 정보	분석에 필요한 세부 정보

Session은 어떻게 정할까?

이번 에이전트 서비스에는 별도의 대화방 개념이 없습니다.

따라서 사용자 ID와 날짜를 조합해 하루 단위로 세션을 만들 수 있습니다.

예를 들어 다음과 같은 형태입니다.

```
user-123_2026-11-02
```

이렇게 하면 한 사용자가 하루 동안 나눈 대화를 하나의 세션으로 묶어서 볼 수 있습니다.

개인정보는 어떻게 관리할까?

트레이스에 원문을 남기면 문제를 정확하게 분석할 수 있지만 개인정보가 외부 관측 도구에 전달된다는 문제가 생깁니다.

예를 들어 다음 정보는 보호해야 합니다.

- 전화번호
- 이메일 주소
- 신용카드 번호
- 기타 개인 식별 정보

따라서 원문을 기록하기로 했다면 다음 사항을 함께 정해야 합니다.

1. 개인정보 패턴을 전송 전에 마스킹합니다.
2. 누가 트레이스에 접근할 수 있는지 정합니다.
3. 데이터 보관 기간을 정합니다.
4. 개인정보처리방침에 외부 관측 도구로 데이터가 전달된다는 사실을 안내합니다.

여기서 중요한 점은 무조건 원문을 기록하거나 무조건 원문을 빼는 것이 아닙니다.

문제 분석에 필요한 정보와 개인정보 보호 사이의 균형을 설계해야 합니다.

8. Langfuse SDK 버전 주의

교안에서는 2026년 10월 기준 Langfuse Python SDK v5를 기준으로 설명하고 있습니다.

인터넷에서 검색되는 예제 중에는 v2, v3 문법이 남아 있을 수 있습니다.

따라서 코딩 에이전트에게 계측 작업을 맡길 때는 다음 사항을 요청해야 합니다.

- Langfuse 공식 최신 문서를 먼저 확인할 것
- 현재 설치된 SDK 버전을 확인할 것
- 공식 문서와 코드의 import 및 함수 이름을 비교할 것
- 참고한 공식 문서의 근거를 제시할 것

AI 코딩 에이전트가 코드를 작성했다고 해서 최신 문법을 사용했다고 단정해서는 안 됩니다.

9. 객관식 퀴즈 정답과 해설

퀴즈 1. 도쿄 날씨 오류의 원인을 찾으려면?

정답: 2번

도구 실행 출력과 답 만들기의 입력·출력을 나란히 비교해야 합니다.

- 도구 실행 결과가 잘못됐는지
- 도구 결과는 정상인데 모델이 잘못 해석했는지

이 두 가지를 구분할 수 있기 때문입니다.

퀴즈 2. 기억 정리를 Generation이 아닌 Span으로 기록하면?

정답: 2번

기억 정리의 모델 이름, 토큰 수, 비용을 확인할 수 없게 됩니다.

Span으로도 실행 시간과 입력·출력 같은 정보를 기록할 수 있지만, Generation으로 기록해야 모델 호출에 관한 정보를 비용 분석에 활용할 수 있습니다.

퀴즈 3. 사용자 메시지 원문을 트레이스에 남긴다면?

정답: 2번

전화번호나 이메일 등을 마스킹하고, 접근 권한·보관 기간·개인정보처리방침을 함께 정해야 합니다.

10. 실제 프로젝트에 적용하기

오늘 배운 계측 구조는 사용자가 준비 중인 AI 서비스에 공통적으로 적용할 수 있습니다.

프로젝트	주요 관측 대상	확인할 문제
AI 친구 에이전트	도구 선택·실행·답변·기억 정리	엉뚱한 답, 느린 응답, 비용 증가
MSM	글 분석·문체 분석·재작성·웹소설 생성	원문 의도와 문체가 유지되는지
뉴스레터 에이전트	뉴스 검색·분류·요약·평가	뉴스 누락, 요약 품질, 토큰 비용
ASP 컨설팅 에이전트	고객 요청 분석·자료 검색·보고서 생성	잘못된 자료 인용, 분석 품질
Travel Creator	여행지 검색·일정 생성·정보 검증	잘못된 장소, 일정 오류, 검색 비용

예를 들어 MSM에서 사용자가 자신의 글을 입력했을 때 다음과 같이 계측할 수 있습니다.

1. 원문 분석: 사용자가 전달한 글의 특징을 분석합니다.
2. 문체 분석: 사용자의 문장 스타일을 파악합니다.
3. 재작성: 원문의 의도와 문체를 유지하면서 정리합니다.
4. 결과 생성: 웹소설 형태로 변환합니다.
5. 기억 정리: 사용자가 선호하는 표현이나 수정 방향을 기록합니다.

이 단계들을 분리해 두면 사용자가 "내 글의 느낌이 사라졌다"고 했을 때 어느 단계에서 원래 문체가 훼손됐는지 찾아볼 수 있습니다.

11. 오늘 반드시 기억해야 할 핵심

오늘의 필수 암기 8가지

1. 서버 로그는 요청의 발생을 보여주지만 내부 처리 과정을 충분히 보여주지는 못합니다.
2. Trace는 한 번의 사용자 메시지 처리 과정을 묶어 보여줍니다.
3. 서로 다른 원인으로 고칠 수 있는 단계는 별도 관찰 단위로 나눕니다.
4. LLM을 호출하는 단계는 Generation, 호출하지 않는 단계는 Span으로 기록합니다.
5. Generation에는 모델 이름과 입력·출력 토큰을 기록해야 비용을 분석할 수 있습니다.
6. 도구 선택·실행·답 만들기를 분리해야 잘못된 답의 원인을 찾을 수 있습니다.
7. User, Session, Environment, Version 등의 속성으로 트레이스를 분류합니다.
8. 원문을 기록할 때는 마스킹, 접근 권한, 보관 기간, 개인정보 안내를 함께 설계합니다.

12. 한 문장으로 오늘 수업 정리

AI 에이전트의 문제를 제대로 분석하려면 한 번의 대화를 Trace로 묶고, 안전장치·도구 선택·도구 실행·답변 생성·기억 정리를 각각 Span 또는 Generation으로 구분해 입력·출력·토큰·비용을 기록해야 합니다.

다음 실습에서 할 일

오늘은 계측 구조를 설계했습니다. 다음 단계에서는 이 설계표를 코딩 에이전트에게 전달해 실제 `app/agent.py`, `app/router.py`, `app/tools.py`, `app/memory.py` 등에 Langfuse 계측 코드를 적용하게 됩니다.

오늘 수업의 가장 중요한 교훈은 이것입니다. 계측은 코딩 에이전트에게 맡길 수 있지만, 무엇을 관찰하고 어떤 정보를 남길지는 서비스의 문제를 이해하는 사람이 먼저 결정해야 합니다.

Chapter05

/강의정리 챕터 05. 요청 틀 쓰는 법 — 코딩 에이전트에게 계측 맡기기

오늘의 핵심 주제

계측 설계도를 코딩 에이전트에게 전달하고, 변경 내용을 검증하는 방법

1. 오늘 수업의 목적

챕터 04에서는 AI 에이전트의 한 번의 대화를 어떤 단계로 나누어 관찰할지 설계했습니다.

오늘은 그 설계도를 실제 코드에 반영하는 방법을 배웁니다.

핵심은 코딩 에이전트에게 단순히 "Langfuse를 연결해 줘"라고 요청하지 않는 것입니다.

무엇을 기록하고, 어떤 이름을 사용하며, 어떤 정보를 남길지 사람이 먼저 결정한 다음 코딩 에이전트에게 작업을 맡겨야 합니다.

오늘 수업은 크게 세 가지입니다.

1. 코딩 에이전트에게 전달할 요청 틀 작성
2. 에이전트가 수정한 코드와 테스트 결과 검증
3. 실제 트레이스를 확인하고 엉뚱한 답의 원인 찾기

2. 왜 요청 틀이 필요한가?

코딩 에이전트에게 다음과 같이 요청한다고 생각해 보겠습니다.

> Langfuse를 우리 서비스에 연결해 줘.

이 요청은 너무 막연합니다.

코딩 에이전트는 두 가지 방향으로 잘못 작업할 수 있습니다.

- LLM 호출만 자동 계측해서 도구 실행 결과는 기록하지 않는 경우
- 모든 함수에 계측을 붙여 트레이스가 지나치게 복잡해지는 경우

첫 번째는 도쿄 날씨 오류의 원인을 찾기 어렵고, 두 번째는 불필요한 기록이 많아져 관리와 비용이 증가할 수 있습니다.

따라서 챕터 04에서 정한 관찰 구조를 명확하게 전달해야 합니다.

AI가 코드를 작성하더라도 무엇을 만들어야 하는지는 사람이 결정해야 합니다.

3. 요청 틀의 구조 이해하기

오늘 교안에서 제공한 요청 틀은 다음과 같은 내용을 포함합니다.

요청 내용	목적
공식 문서 확인	오래된 SDK 문법 사용 방지
버전 고정	버전 변경으로 인한 오류 방지
관찰 단위 지정	챕터 04의 설계 유지
토큰과 비용 기록	모델별 비용 분석

요청 내용	목적
입력과 출력 기록	문제 발생 원인 분석
개인정보 마스킹	개인정보 보호
키가 없으면 계측 끄기	기존 서비스 실행 안정성
기존 테스트 통과	기존 기능 보호
변경 파일과 이유 설명	코드 변경에 대한 사람의 검토

이 중에서 특히 중요한 것은 빈칸을 코딩 에이전트가 아니라 서비스 운영자가 결정해야 한다는 점입니다.

예를 들어 SDK 버전과 개인정보 원문 기록 여부는 서비스 운영 정책과 연결되므로 사람이 결정해야 합니다.

4. 요청 틀에서 반드시 이해해야 할 내용

① 공식 문서를 먼저 확인하게 하기

Langfuse Python SDK는 버전별로 사용법이 달라질 수 있습니다.

인터넷에 있는 오래된 예제를 그대로 사용하면 현재 버전에서 동작하지 않을 수 있습니다.

따라서 코딩 에이전트에게 다음을 요청합니다.

- 공식 최신 문서를 먼저 확인
- 참고한 문서 주소 제시
- 실제 설치할 SDK 버전 확인
- 공식 문법에 맞게 코드 작성

여기서 중요한 것은 AI가 작성한 코드를 무조건 신뢰하지 않는 것입니다.

코딩 에이전트가 제시한 근거를 사람이 확인하는 과정이 필요합니다.

② SDK 버전 고정하기

예를 들어 다음과 같이 버전을 고정합니다.

```
requirements.txt

langfuse==[확인한 버전]
```

버전을 고정하는 이유는 개발 환경과 배포 환경에서 서로 다른 SDK가 설치되는 일을 방지하기 위해서입니다.

이는 앞서 배운 Docker 이미지와 배포 버전 관리의 개념과도 연결됩니다.

③ 관찰 단위와 이름을 그대로 사용하기

챕터 04에서 설계한 이름을 그대로 전달합니다.

이름	종류
`chat-turn`	Trace
`safety`	Span
`choose-tools`	Generation
`tool:<도구 이름>`	Span
`reply`	Generation
`memory-update`	Generation
`memory-summary`	Generation
`greeting`	Trace

이름을 임의로 바꾸면 나중에 Langfuse 화면에서 단계별 데이터를 비교하기 어려워집니다.

예를 들어 어떤 트레이스에서는 `reply`, 다른 트레이스에서는 `generate-answer`라고 기록하면 답변 생성 단계만 모아서 분석하기가 번거로워집니다.

④ 모델 호출과 비용 기록하기

Generation에는 다음 정보를 남깁니다.

- 모델 이름
- 입력 토큰 수
- 출력 토큰 수

이 세 가지가 있어야 모델별 사용량과 비용을 분석할 수 있습니다.

특히 도구 선택 단계가 규칙 기반으로 실행되면 실제 LLM 호출이 없으므로 토큰은 0입니다.

따라서 `decided_by` 속성에 모델이 결정했는지 규칙이 결정했는지를 함께 기록해야 합니다.

⑤ 개인정보 원문과 마스킹 정책

이번 실습에서는 원문을 남기기로 합니다.

다만 다음 정보는 전송 전에 가립니다.

- 전화번호
- 이메일

- 카드 번호

원문을 남기는 이유는 문제의 원인을 정확히 분석하기 위해서입니다.

예를 들어 도쿄 날씨 질문에서 도구가 실패했다는 결과를 모델이 받았는지 확인하려면 입력과 출력 내용을 볼 수 있어야 합니다.

하지만 개인정보를 무조건 수집해서는 안 됩니다.

원문 기록은 접근 권한, 보관 기간, 개인정보처리방침과 함께 관리해야 합니다.

5. 코딩 에이전트에게 작업을 맡기는 순서

실제 작업 절차

1. 저장소 폴더에서 코딩 에이전트 실행

   Claude Code 등 코딩 에이전트를 프로젝트 루트에서 실행합니다.

2. 요청 틀의 빈칸 채우기

   SDK 버전과 원문 기록 여부 등 사람이 결정할 내용을 입력합니다.

3. 변경 계획 검토

   에이전트가 먼저 제시하는 공식 문서 주소, 수정 파일, 수정 이유를 확인합니다.

4. 코드 변경 승인

   계획이 적절한지 확인한 뒤 실제 코드 수정을 진행하게 합니다.

5. 테스트 결과 확인

   기존 테스트가 모두 통과하는지 확인합니다.

6. 실제 서비스 실행

   로컬에서 에이전트를 실행하고 Langfuse에 트레이스가 기록되는지 확인합니다.

승인 전에 확인할 다섯 가지

확인 사항	확인 이유
공식 문서와 import·함수 이름이 일치하는가?	구버전 문법 방지
챕터 04의 관찰 단위 이름이 그대로인가?	분석 기준 유지
API 키가 없으면 계측을 끄는가?	기존 서비스 보호
서버 로그에 대화 원문을 새로 남기지 않는가?	개인정보 보호
키가 코드나 테스트 파일에 들어가지 않았는가?	비밀정보 보호

추가로 중요한 사항이 있습니다.

이 프로젝트의 LLM 호출은 OpenAI 호환, Claude, Fake LLM 세 가지 방식으로 나뉩니다.

특정 제공사에만 맞춘 자동 계측을 적용하면 다른 제공사로 변경했을 때 트레이스가 누락될 수 있습니다.

따라서 세 가지 LLM 구현 방식 모두 동일한 관찰 구조로 기록되는지 확인해야 합니다.

6. 실제 실행 명령어 이해하기

코드 변경이 끝나면 다음 명령어를 실행합니다.

```
# requirements.txt에 지정된 패키지 설치
pip install -r requirements.txt

# 기존 테스트 실행
pytest -q

# 로컬 서버 실행
uvicorn app.main:app --reload --port 8080
```

각 명령어의 의미는 다음과 같습니다.

명령어	역할
`pip install -r requirements.txt`	필요한 Python 패키지 설치
`pytest -q`	기존 기능이 정상적으로 작동하는지 테스트
`uvicorn app.main:app`	AI 에이전트 서버 실행
`--reload`	코드 변경 시 자동 재시작
`--port 8080`	8080 포트에서 서버 실행

여기서 `pytest -q`는 특히 중요합니다.

Langfuse를 연결했다고 기존 AI 친구 서비스의 기능이 망가지면 안 되기 때문입니다.

새로운 기능을 추가할 때는 기존 기능이 그대로 작동하는지 반드시 확인해야 합니다.

7. Langfuse에서 실제 트레이스 확인하기

로컬 서버가 실행되면 브라우저에서 다음 주소로 접속합니다.

`http://localhost:8080`

캐릭터를 저장하고 평범한 잡담을 두세 개 입력합니다.

그다음 Langfuse에서 다음 순서로 확인합니다.

1. Langfuse 프로젝트에 접속합니다.
2. Tracing → Traces로 이동합니다.
3. 보낸 메시지 수만큼 `chat-turn` 트레이스가 생성됐는지 확인합니다.
4. 트레이스 하나를 열어 왼쪽 단계 구조를 확인합니다.
5. `reply`를 눌러 모델 이름, 입력·출력 토큰, 비용을 확인합니다.

Observability & Tracing for Inferable (open source) - Langfuse

langfuse.com

잡담에는 도구를 사용하지 않으므로 `tool:...` 단계가 없는 것이 정상입니다.

반면 처음 대화를 시작했다면 `greeting` 트레이스가 별도로 생성될 수 있습니다.

이렇게 해서 실제 코드에 넣은 계측이 의도한 대로 작동하는지 확인합니다.

8. 도쿄 날씨 오류를 직접 재현하고 원인 찾기

오늘 실습의 핵심입니다.

이제 동료가 경험한 문제를 직접 재현합니다.

① 문제 상황 만들기

AI 친구 에이전트에 다음 두 가지 질문을 입력합니다.

- "도쿄 날씨 어때?"
- "내일 서울 비 와?"

서울은 지원되는 지역이고 도쿄는 지원되지 않는 지역이므로 두 결과를 비교할 수 있습니다.

② 도쿄 트레이스를 거슬러 올라가기

다음 순서로 확인합니다.

오류 원인 추적 순서

1. reply 출력

   사용자에게 실제로 어떤 답을 전달했는지 확인합니다.

2. reply 입력

   모델이 답변을 만들기 전에 어떤 정보를 받았는지 확인합니다.

3. tool:get_weather 출력

날씨 도구가 어떤 결과를 반환했는지, 성공했는지 확인합니다.

4. choose-tools 출력

   어떤 도구를 어떤 인자로 선택했는지, 누가 결정했는지 확인합니다.

5. 서울 트레이스와 비교

   두 트레이스가 처음 달라지는 단계를 찾아냅니다.

③ 결과 해석하기

도쿄 트레이스에서 다음과 같은 상황을 가정해 보겠습니다.

- 도구 선택: `get_weather` 선택
- 도구 실행: 지원하지 않는 지역이라는 실패 결과 반환
- 답 만들기 입력: 도구 실패 결과가 정상적으로 전달됨
- 답 만들기 출력: 실제 날씨를 지어내서 답변

이 경우 문제는 도구 선택이나 도구 실행이 아닙니다.

도구 실행 결과는 올바르게 전달됐지만, 답변 생성 단계에서 모델이 실패 정보를 무시한 것입니다.

따라서 답 만들기 프롬프트를 수정해야 합니다.

예를 들어 다음과 같은 규칙을 추가할 수 있습니다.

> 도구 실행이 실패하거나 필요한 정보를 가져오지 못했다면, 추측하거나 사실처럼 만들어 답하지 말고 실패 사실을 사용자에게 알린다.

반대로 도구가 잘못된 지역을 선택했다면 도구 선택 프롬프트나 규칙을 수정해야 합니다.

중요한 것은 오류가 발생한 마지막 단계가 아니라, 처음 어긋난 단계를 찾는 것입니다.

LLM은 같은 입력에도 다른 답변을 만들 수 있으므로 여러 번 실행해 보는 것이 좋습니다. 한 번의 결과만으로 모델의 동작을 단정하지 않습니다.

9. 오늘 배운 내용을 실제 프로젝트에 적용한다면?

오늘 수업은 MSM이나 ASP 같은 서비스의 품질을 높이는 데도 직접 활용할 수 있습니다.

프로젝트	추적할 단계	발견할 수 있는 문제
MSM	원문 분석 → 문체 분석 → 재작성 → 웹소설 생성	원래 문체가 어느 단계에서 사라졌는가?

프로젝트	추적할 단계	발견할 수 있는 문제
뉴스레터 에이전트	검색 → 분류 → 요약 → 평가	뉴스가 누락되거나 잘못 요약된 이유는?
ASP 컨설팅 에이전트	자료 검색 → 분석 → 전략 도출 → 보고서 작성	잘못된 자료가 어느 단계에서 결론에 반영됐는가?
Travel Creator	여행지 검색 → 정보 확인 → 일정 생성	잘못된 장소나 일정이 어느 단계에서 만들어졌는가?

예를 들어 MSM에서 사용자가 "내 글의 느낌이 사라졌다"고 평가했다면 다음을 비교할 수 있습니다.

- 원문 분석 단계의 입력과 출력
- 문체 분석 단계의 입력과 출력
- 재작성 단계의 입력과 출력
- 최종 웹소설 생성 단계의 입력과 출력

이렇게 하면 단순히 결과가 마음에 들지 않는다는 사실을 넘어, 어느 단계의 프롬프트나 처리 로직을 고쳐야 하는지 판단할 수 있습니다.

10. 오늘 반드시 기억해야 할 핵심

필수 암기 8가지

1. 코딩 에이전트에게 작업을 맡기기 전에 계측 설계를 사람이 먼저 정합니다.
2. 최신 공식 문서와 SDK 버전을 확인하고 버전을 고정합니다.
3. 관찰 단위의 이름과 종류는 미리 정한 설계표를 그대로 따릅니다.
4. OpenAI 호환·Claude·Fake LLM 세 가지 모두 계측되는지 확인합니다.
5. 개인정보는 필요한 범위에서 기록하고 전송 전에 마스킹합니다.
6. API 키가 없을 때도 기존 서비스가 정상 작동하도록 만듭니다.
7. 기존 테스트를 통과한 뒤 실제 Langfuse 화면에서 트레이스를 확인합니다.
8. 오류가 발생하면 트레이스의 입력과 출력을 거슬러 올라가 처음 어긋난 단계를 찾습니다.

11. 주관식 퀴즈 모범 답안 작성 방법

질문: 내 컴퓨터에서 만든 도쿄 트레이스에서 처음 어긋난 단계는 어디였나요?

실습 결과에 따라 답이 달라지므로, 아래는 도쿄 날씨를 지어내는 문제가 재현됐을 때의 예시입니다.

> 처음 어긋난 단계는 `reply` 입니다. `tool:get_weather` 의 출력에서 지원하지 않는 지역이라는 실패 결과가 확인됐고, 이 내용이 `reply` 의 입력에도 전달됐습니다. 그런데 `reply` 의 출력에서는 실제 날씨를 확인한 것처럼 답변을 만들어 냈습니다. 따라서 도구 실행 코드를 수정하기보다는, 도구 실패 시 추측하지 않고 사용자에게 실패 사실을 알리도록 답변 생성 프롬프트를 개선해야 합니다.

실제 실습에서 문제가 재현되지 않았다면 다음과 같이 작성하면 됩니다.

> 이번 실행에서는 문제가 재현되지 않았습니다. 도구 실행 결과와 답변이 정상적으로 연결되어 있었으며, LLM의 비결정성 때문에 같은 질문을 여러 번 실행해 추가 확인할 필요가 있습니다.

12. 한 문장으로 오늘 수업 정리

코딩 에이전트에게 계측 작업을 맡길 때는 사람이 먼저 관찰 구조와 개인정보 정책을 결정하고, 변경된 코드를 테스트한 뒤 Langfuse 트레이스의 입력과 출력을 비교해 문제의 최초 발생 단계를 찾아야 합니다.

오늘의 가장 중요한 교훈: AI 코딩 에이전트는 코드를 대신 작성할 수 있지만, 무엇을 관찰하고 어떤 기준으로 문제를 판단할지는 서비스 책임자가 결정해야 합니다.

Chapter06

/강의정리 챕터 06. 가끔 느린 답과 예상보다 많은 요금

1. 학습 목적

이번 챕터의 핵심은 AI 에이전트가 느려지는 원인과 예상보다 많은 비용이 발생하는 원인을 Langfuse로 찾아내는 것입니다.

챕터 05에서 AI 에이전트의 각 작업을 관찰할 수 있도록 계측했다면, 이번에는 그 데이터를 활용해 두 가지 문제를 분석합니다.

1. 사용자가 답변을 기다리는 시간이 왜 길어지는가?
2. 메시지 수보다 실제 AI 사용 요금이 왜 더 많이 나오는가?

즉, 단순히 AI가 느리다거나 비싸다는 현상을 확인하는 것이 아니라, 어느 단계에서 시간이 걸리고 어느 단계에서 비용이 발생하는지 찾아내는 과정입니다.

2. 중학생도 이해하는 핵심 개념

① 전체 응답 시간과 사용자가 기다리는 시간은 다릅니다

AI 친구에게 질문하면 다음과 같은 순서로 작업이 진행됩니다.

AI 친구의 응답 과정

1. choose-tools (도구 선택)

   질문을 이해하고 날씨 조회 같은 도구를 사용할지 결정합니다.

2. tool execution (도구 실행)

   외부 날씨 API 등을 호출하고 결과를 가져옵니다.

3. reply (답변 생성)

   결과를 바탕으로 답변을 만들고 첫 글자를 사용자에게 보여줍니다.

4. memory-update (기억 정리)

   답변을 보낸 뒤 대화 내용을 기억에 반영합니다.

사용자가 기다리는 시간

첫 글자가 나올 때까지

choose-tools + tool 실행 + reply 첫 토큰까지의 시간

Langfuse 전체 지연 시간

요청 전체 시간

기억 정리까지 포함한 전체 처리 시간

예를 들어 답변이 화면에 나오기까지 5초가 걸리고, 답변이 나온 뒤 기억 정리에 3초가 걸렸다면 다음과 같습니다.

- 사용자가 체감하는 대기 시간: 약 5초
- Langfuse의 전체 지연 시간: 약 8초

따라서 전체 지연 시간이 길다고 해서 반드시 사용자가 답을 기다리는 시간도 긴 것은 아닙니다.

중요한 개념: Time to First Token

사용자가 질문을 보낸 뒤 AI가 첫 글자를 화면에 보여줄 때까지 걸리는 시간입니다. 사용자가 느끼는 응답 속도를 파악할 때 중요한 지표입니다.

② 느린 원인은 단계별로 찾아야 합니다

Langfuse의 Tracing에서 Latency를 기준으로 정렬하면 가장 느린 트레이스를 찾을 수 있습니다.

관찰 단계	느려지는 원인	개선 아이디어
choose-tools	도구 선택 모델 응답 지연	단순한 질문은 규칙으로 처리
tool execution	외부 API 응답 지연	결과 캐싱, API 최적화
reply	답변 생성 모델 지연	모델 변경, 입력 토큰 축소
memory-update	기억 정리 작업 지연	비동기 처리, 기억 정리 최적화

여기서 중요한 것은 가장 빠른 트레이스와 가장 느린 트레이스를 비교하는 것입니다.

한 번 느렸다는 사실만으로 원인을 단정하면 안 됩니다. LLM 제공사의 응답 속도도 시간대에 따라 달라질 수 있기 때문입니다.

3. 메시지 수보다 AI 요금이 더 많이 나오는 이유

이번 챕터에서 특히 중요한 내용입니다.

많은 초보자는 다음과 같이 계산합니다.

> 하루 메시지 1,000건 × AI 호출 1회 비용 = 하루 예상 요금

하지만 실제 AI 에이전트는 메시지 하나를 처리하면서 모델을 여러 번 호출할 수 있습니다.

① 메시지 하나에 여러 번 모델을 호출합니다

일반적인 메시지의 모델 호출 구조는 다음과 같습니다.

일반적인 메시지 1건

① choose-tools

도구를 사용할지 판단하는 모델 호출

② reply

사용자에게 답변을 생성하는 모델 호출

③ memory-update

대화 내용을 기억에 반영하는 모델 호출

요약 조건에 해당하면 memory-summary 호출이 추가될 수 있습니다.

여기에 다음과 같은 추가 호출도 있습니다.

- 대화가 처음 시작되거나 6시간 넘게 쉬었다가 화면을 열면 greeting 호출
- 요약되지 않은 메시지가 40개를 넘으면 memory-summary 호출

따라서 메시지 1건에 모델 호출 3회가 발생할 수 있고, 상황에 따라 더 늘어날 수 있습니다.

② 대화가 길어지면 비용도 증가합니다

답변을 만들 때 AI에게 전달하는 입력에는 여러 정보가 들어갑니다.

- 캐릭터 지시문
- 기억 메모
- 대화 요약
- 최근 10턴의 대화

대화가 길어질수록 AI가 읽어야 하는 입력 토큰이 늘어납니다.

예를 들어 똑같이 "오늘 날씨 어때?"라고 질문해도 대화가 짧을 때와 100번 이상 대화한 뒤에는 답변 생성에 필요한 입력 토큰이 달라질 수 있습니다.

핵심은 AI 사용료가 질문의 길이만으로 결정되는 것이 아니라, AI에게 전달하는 전체 입력과 생성하는 출력, 호출 횟수에 따라 달라진다는 것입니다.

③ Langfuse로 비용을 분석하는 방법

확인 위치	확인할 내용
Tracing > Traces	Cost 기준으로 비싼 트레이스 찾기
개별 Trace	choose-tools, reply, memory-update 비용 비교
Generation	모델명, 입력 토큰, 출력 토큰
Dashboards	하루 전체 트레이스 수, 총비용, 모델별 비용

하루 평균 메시지 비용은 다음과 같이 계산합니다.

$$
\text{메시지당 평균 비용}
=
\frac{\text{하루 총 AI 비용}}{\text{하루 메시지 수}}
$$

다만 이 수치는 실제 사용자의 메시지와 트레이스가 충분히 쌓인 뒤 계산해야 의미가 있습니다.

4. 실제 프로젝트에 적용한다면

사용자가 추진 중인 AI 서비스에도 직접 연결되는 내용입니다.

프로젝트	느린 응답 분석	비용 분석
AI 친구 에이전트	도구 선택, 날씨 API, 답변 생성 중 지연 단계 확인	기억 정리와 요약 호출 비용 확인
MSM	원문 분석, 문체 분석, 웹소설 생성 중 지연 확인	원문 분석과 장문 생성의 토큰 비용 비교
ASP 컨설팅 에이전트	문서 검색, RAG, 답변 생성 중 지연 확인	문서 검색 후 답변 생성에 드는 비용 확인
뉴스레터 에이전트	뉴스 수집, 요약, 평가 단계별 지연 확인	기사별 요약과 평가 모델 호출 비용 확인
Travel Creator	장소 검색, 일정 생성, 외부 API 지연 확인	검색 횟수와 일정 생성 비용 확인

MSM에 적용하는 예시

MSM에서 사용자가 자신의 일기를 입력하고 웹소설로 변환한다고 가정하겠습니다.

1. 원문 분석: 글의 주제와 감정 파악
2. 문체 분석: 사용자의 말투와 표현 습관 파악
3. 스토리 구성: 사건과 인물의 흐름 정리
4. 웹소설 생성: 실제 이야기 작성
5. 기억 업데이트: 사용자의 선호와 설정 저장

이때 웹소설 생성이 느린지, 원문 분석이 느린지, 기억 정리가 불필요하게 반복되는지 구분할 수 있습니다.

비용도 마찬가지입니다. 사용자가 글을 한 번 입력했을 때 실제로 몇 번 모델을 호출하고 어느 단계에서 가장 많은 토큰을 사용하는지 확인할 수 있습니다.

이런 데이터가 쌓이면 MSM의 무료 이용량, 유료 구독료, 글 생성 횟수 등을 결정하는 근거가 됩니다.

5. AI 창업자가 반드시 알아야 하는 이유

이번 강의의 핵심은 단순한 기술 모니터링이 아닙니다.

AI 서비스의 운영비와 사용자 경험을 관리하는 방법을 배우는 것입니다.

AI 서비스를 사업으로 운영할 때는 다음 세 가지를 함께 관리해야 합니다.

- 품질: 답변이 정확하고 유용한가?
- 속도: 사용자가 기다리는 시간이 적절한가?
- 비용: 한 명의 고객을 서비스하는 데 얼마가 드는가?

예를 들어 월 구독료가 9,900원인 서비스를 만들었는데, 고객 한 명의 월평균 AI 사용 비용이 12,000원이라면 서비스를 많이 팔수록 손해가 발생할 수 있습니다.

따라서 AI 사업자는 기능을 만드는 것뿐 아니라 고객 한 명당 사용 비용(Unit Economics)을 이해해야 합니다.

6. 객관식 퀴즈 정답 및 해설

정답 2번

트레이스 하나를 열어 모델을 부르는 관찰 단위가 몇 개이고 각각의 토큰과 비용이 얼마인지, 그리고 greeting처럼 메시지 없이 생긴 트레이스가 있는지 확인해야 합니다.

이유는 메시지 하나가 여러 번 모델을 호출할 수 있고, 사용자가 메시지를 보내지 않아도 greeting 같은 별도 호출이 발생할 수 있기 때문입니다.

- 1번: 트레이스 수가 메시지 수와 다를 수 있어 호출 비용 원인을 설명하지 못합니다.
- 3번: 지연 시간은 속도 문제를 분석하는 지표입니다.
- 4번: 점수는 품질 평가에 활용하지만 비용 증가의 직접 원인을 찾는 첫 단계는 아닙니다.

7. 오늘 반드시 기억할 내용

1. Langfuse 전체 Latency와 사용자가 체감하는 대기 시간은 다릅니다.
2. 사용자가 기다리는 시간은 첫 글자가 화면에 나올 때까지인 Time to First Token이 중요합니다.
3. 느린 원인은 choose-tools, tool execution, reply 등 단계별로 비교해야 합니다.
4. 메시지 한 건이 모델을 여러 번 호출할 수 있습니다.
5. 기억 정리와 greeting도 AI 비용을 발생시킬 수 있습니다.
6. 대화가 길어지면 입력 토큰이 증가해 같은 질문도 비용이 커질 수 있습니다.
7. AI 서비스의 사업성을 판단하려면 고객 한 명당 평균 AI 비용을 알아야 합니다.

8. 한 문장으로 정리

Langfuse는 AI가 왜 느린지, 어디서 비용이 발생하는지를 단계별로 보여주며, 이를 통해 AI 서비스의 품질·속도·수익성을 관리할 수 있게 해주는 도구입니다.

Chapter07

/강의정리 챕터 07. 속성이 있어야 보이는 두 상황

1. 학습 목적

이번 챕터의 핵심은 AI 에이전트의 사용자를 구분하고, 배포 버전별로 성능을 비교할 수 있도록 Langfuse에 속성(Attribute)을 추가하는 것입니다.

앞선 챕터에서는 AI 에이전트가 어떤 단계에서 느려지는지, 어느 단계에서 비용이 발생하는지 확인했습니다.

하지만 실제 서비스 운영에서는 두 가지 문제가 더 생깁니다.

1. 특정 사용자가 이야기한 내용을 AI가 기억하지 못한다면 어디서 문제가 발생했는가?
2. 프로그램을 업데이트한 뒤 응답이 느려졌다면 업데이트가 원인인가?

이 문제를 해결하려면 트레이스에 사용자와 세션, 실행 환경, 배포 버전 등의 정보가 있어야 합니다.

2. 중학생도 이해하는 핵심 개념

① 속성(Attribute)이란 무엇인가?

속성은 트레이스에 붙이는 일종의 이름표입니다.

택배 상자에 받는 사람, 배송 주소, 배송 날짜를 붙여 놓으면 상자를 쉽게 구분할 수 있듯이, AI 트레이스에도 속성을 붙이면 누구의 대화인지, 어느 환경에서 실행됐는지, 어떤 버전에서 발생했는지 구분할 수 있습니다.

이번 챕터에서 추가하는 주요 속성은 다음과 같습니다.

속성	쉬운 설명	예시
user_id	누가 사용했는가?	사용자 ID
session_id	어떤 대화 묶음인가?	사용자 ID + 한국 날짜
environment	어디서 실행했는가?	local, staging, production
version	어떤 배포 버전인가?	Cloud Run 리비전
metadata	추가적인 상세 정보	커밋 SHA, 도구, 확신도
tags	검색과 분류를 위한 꼬리표	weather, safety-response

여기서 특히 중요한 것은 `user_id`, `session_id`, `environment`, `version`입니다.

- `user_id`: 사용자별 대화와 비용을 확인합니다.
- `session_id`: 한 사용자의 특정 대화 흐름을 연결합니다.
- `environment`: 로컬, 테스트, 실제 운영 환경을 구분합니다.
- `version`: 어느 배포 버전에서 문제가 발생했는지 확인합니다.

핵심은 트레이스 자체에 충분한 정보를 붙여야 나중에 원하는 기준으로 검색하고 비교할 수 있다는 것입니다.

② 속성은 한 번만 설정하면 됩니다

강의에서 중요한 부분은 속성을 각 단계마다 반복해서 설정하지 않아도 된다는 점입니다.

한 번의 사용자 메시지를 처리하는 `chat-turn` Trace의 시작 부분에서 속성을 설정하면 그 안에 있는 Span과 Generation에도 함께 적용할 수 있습니다.

예를 들어 다음과 같습니다.

chat-turn Trace

user_id: user123

session_id: user123-2026-11-02

environment: staging

version: friend-agent-00002

safety Span

choose-tools Generation

tool execution Span

reply Generation

memory-update Generation

하나의 Trace에 붙인 공통 속성을 하위 관찰 단위에서도 활용할 수 있습니다.

이렇게 하면 나중에 특정 사용자의 모든 대화나 특정 버전의 실행 기록을 쉽게 찾을 수 있습니다.

3. 첫 번째 문제: AI가 사용자의 기억을 잊어버리는 경우

예를 들어 사용자가 AI 친구에게 다음과 같이 말합니다.

- 월요일: "나 다음 주 토요일에 이사 가."

- 화요일: 잡담
- 수요일: "내가 다음 주에 뭐 한다고 했지?"

AI가 이사 이야기를 기억하지 못한다면 세 가지 가능성을 확인해야 합니다.

기억이 끊기는 세 가지 지점

문제 1

기억 저장 실패

`memory-update` 출력에 이사 정보가 추가되지 않았습니다.

문제 2

기억 전달 실패

기억은 저장됐지만 `reply` 입력에 포함되지 않았습니다. 기억 메모는 최대 30개까지만 들어갑니다.

문제 3

기억 활용 실패

`reply` 입력에 이사 정보가 들어갔는데도 AI가 답변에 반영하지 않았습니다. 이 경우 답변 생성 프롬프트를 확인해야 합니다.

여기서 주의할 점이 하나 있습니다.

같은 날 몇 마디 대화하는 동안에는 최근 10턴의 원문이 답변 생성에 포함될 수 있습니다. 따라서 AI가 이사 이야기를 맞혔다고 해서 장기 기억에 제대로 저장되었다고 판단하면 안 됩니다.

진짜 기억 능력을 확인하려면 며칠 뒤에도 기억하는지 확인해야 합니다.

이를 위해 Langfuse에서 다음 순서로 확인합니다.

1. Users 화면에서 특정 사용자를 찾습니다.
2. Sessions 화면에서 해당 날짜의 대화를 찾습니다.
3. 이사 이야기를 한 트레이스의 `memory-update` 출력을 확인합니다.
4. 나중에 질문한 트레이스의 `reply` 입력에 기억이 포함됐는지 확인합니다.
5. 기억이 입력됐는데도 답을 못 했다면 답변 생성 과정을 분석합니다.

4. 두 번째 문제: 업데이트한 뒤 서비스가 느려진 경우

이번에는 배포 버전별로 성능을 비교하는 방법입니다.

① Local, Staging, Production 구분

환경	의미	사용 목적
local	내 컴퓨터	개발 및 실험
staging	실제 운영과 유사한 테스트 서버	배포 전 검증
production	실제 고객이 사용하는 서비스	정식 운영

예를 들어 내 컴퓨터에서 테스트한 트레이스와 실제 테스트 서버에서 발생한 트레이스가 한 목록에 섞여 있으면 정확한 비교가 어렵습니다.

따라서 `environment` 속성으로 구분합니다.

② Version으로 배포 전후 비교

Cloud Run에서는 배포할 때마다 새로운 리비전(Revision)이 만들어집니다.

예를 들어 다음과 같습니다.

Cloud Run 배포 버전 비교

리비전 A

`friend-agent-00001`

커밋 A / 응답 평균 2초

코드 업데이트 및 재배포

리비전 B

`friend-agent-00002`

커밋 B / 응답 평균 5초

위 숫자는 이해를 위한 예시입니다. 실제 성능은 Langfuse에서 측정해야 합니다.

두 버전에 비슷한 메시지를 보내고 `Version`으로 트레이스를 나눠 비교하면 됩니다.

- 두 버전의 지연 시간이 비슷하다면 업데이트가 원인이라고 단정하기 어렵습니다.
- 새 버전이 더 느리다면 Latency와 reply의 첫 글자 출력 시간을 비교합니다.
- 어느 단계가 느려졌는지 확인한 뒤 metadata의 커밋 SHA로 변경 내용을 추적합니다.

이 방식은 일부 사용자에게만 새 버전을 보내는 카나리 배포에서도 유용합니다.

5. Staging에 Langfuse 연결하기

실제 테스트 서버에서도 트레이스를 수집하려면 Langfuse 키를 Cloud Run에 넣어야 합니다.

다만 비밀키를 일반 환경변수나 코드에 직접 넣으면 보안 문제가 발생할 수 있습니다.

따라서 Google Cloud의 Secret Manager를 활용합니다.

연결 구조

Google Cloud Secret Manager

`langfuse-secret-key`

비밀키 보관

서비스 계정에 읽기 권한 부여

Cloud Run: friend-agent-staging

공개키 + 접속 주소 + Secret Manager 비밀키

Langfuse Cloud

Staging 트레이스 수집

주요 명령어는 다음과 같습니다.

```
gcloud secrets add-iam-policy-binding langfuse-secret-key --member "serviceAccount:fri
end-agent-runtime@<PROJECT_ID>.iam.gserviceaccount.com" --role roles/secretmanager.sec
retAccessor
```

```
gcloud run services update friend-agent-staging --region us-central1 \
  --update-env-vars LANGFUSE_PUBLIC_KEY=<공개키>,LANGFUSE_BASE_URL=https://us.cloud.lan
gfuse.com \
  --update-secrets LANGFUSE_SECRET_KEY=langfuse-secret-key:latest
```

여기서 꼭 기억할 점은 다음과 같습니다.

- 비밀키는 Secret Manager에 보관합니다.
- 서비스 계정에는 필요한 시크릿 하나만 읽을 권한을 줍니다.
- `-update-env-vars`는 기존 환경변수를 유지하면서 필요한 값을 추가·수정합니다.
- `-set-env-vars`는 기존 환경변수 구성을 대체할 수 있으므로 주의해야 합니다.
- Windows PowerShell에서는 줄 연결에 백틱을 사용하거나 한 줄로 작성합니다.

6. Cloud Run에서 트레이스가 늦게 나타나는 이유

Langfuse SDK는 데이터를 매번 즉시 보내지 않고 일정량을 모아 전송할 수 있습니다.

그런데 Cloud Run은 요청 처리 중일 때와 요청이 끝난 뒤 CPU 사용 조건이 달라질 수 있습니다.

그래서 트레이스가 늦게 나타나거나 일부가 누락되는 문제가 생길 수 있습니다.

이때 사용할 수 있는 것이 `flush()` 입니다.

쉽게 말해 아직 보내지 않고 모아 둔 데이터를 지금 전송하라고 지시하는 명령입니다.

구분	의미
자동 전송	SDK가 일정량의 데이터를 모아 전송
flush()	모아 둔 데이터를 명시적으로 전송
주의점	응답 종료 시 flush하면 응답이 조금 늦어질 수 있음

데이터가 누락되지 않도록 하는 것과 사용자 응답 속도를 유지하는 것 사이에서 적절한 균형이 필요합니다.

7. 실제 프로젝트에 적용한다면

이번 챕터는 특히 여러 사용자가 이용하는 AI 서비스를 운영할 때 중요합니다.

프로젝트	적용 방법
AI 친구 에이전트	사용자별 대화와 기억 실패 원인 추적
MSM	사용자별 문체·스토리 기억 관리, 모델 버전별 결과 비교
ASP 컨설팅 에이전트	고객사별 분석 기록과 버전별 컨설팅 품질 비교
뉴스레터 에이전트	배포 버전별 뉴스 요약 품질과 비용 비교
Travel Creator	사용자별 여행 취향 기억과 일정 생성 품질 추적

특히 MSM에서는 사용자마다 다른 문체와 인생 이야기를 기억해야 합니다.

`user_id` 와 `session_id` 를 활용하면 특정 사용자의 글쓰기 이력과 기억이 어느 단계에서 끊겼는지 확인할 수 있습니다.

ASP 컨설팅 에이전트에서는 고객사별로 기록을 구분하고, 새 프롬프트나 모델을 적용한 뒤 컨설팅 결과가 개선됐는지 비교할 수 있습니다.

8. AI 창업자가 반드시 알아야 하는 이유

이번 챕터는 AI 서비스가 실제 고객을 만나기 시작했을 때 필요한 운영 역량을 다룹니다.

개발 단계에서는 AI가 잘 작동하는지만 확인하면 되지만, 사업 단계에서는 다음 질문에 답할 수 있어야 합니다.

1. 어떤 고객에게 문제가 발생했는가?
2. 어떤 대화에서 문제가 시작됐는가?
3. 어떤 배포 버전에서 문제가 발생했는가?

4. 업데이트 전후 품질과 속도는 어떻게 달라졌는가?
5. 문제가 발생한 고객에게 어떤 개선 조치를 해야 하는가?

이러한 질문에 데이터로 답할 수 있어야 고객 신뢰를 유지하고 서비스를 지속적으로 개선할 수 있습니다.

9. 오늘 반드시 기억할 내용

1. `user_id`는 사용자를 구분합니다.
2. `session_id`는 한 사용자의 대화 흐름을 연결합니다.
3. `environment`는 local, staging, production을 구분합니다.
4. `version`은 Cloud Run 리비전을 구분합니다.
5. 기억 실패는 저장 실패, 전달 실패, 활용 실패로 나눠 확인합니다.
6. 배포 전후 비교는 비슷한 메시지를 보내고 Version별로 확인해야 합니다.
7. 비밀키는 Secret Manager에 보관하고 서비스 계정에 최소 권한만 줍니다.
8. `flush()`는 모아 둔 트레이스 데이터를 전송하는 명령입니다.

10. 한 문장으로 정리

Langfuse에 사용자·세션·환경·버전 속성을 추가하면 누구에게 어떤 문제가 발생했는지, 그리고 업데이트 전후 서비스가 어떻게 달라졌는지를 데이터로 추적할 수 있습니다.

이번 챕터의 학습 연결: 챕터 04에서 관찰 구조를 설계하고, 챕터 05에서 실제 코드에 계측을 적용하고, 챕터 06에서 속도와 비용을 분석했다면, 챕터 07에서는 사용자와 배포 버전을 구분해 문제를 추적하는 단계까지 발전한 것입니다. 즉, AI 에이전트를 단순히 만드는 수준에서 실제 고객이 사용하는 서비스를 운영하는 수준으로 넘어가고 있습니다.

Chapter08

/강의정리 챕터 08. 언제였는지 모르는 별로인 답

1. 학습 목적

이번 챕터의 핵심은 사용자가 AI 답변에 남긴 반응을 Langfuse 점수(Score)로 기록해, 문제가 있었던 답변을 쉽게 찾아내는 것입니다.

앞선 챕터에서는 사용자별로 트레이스를 구분하고, 배포 버전별로 성능을 비교할 수 있도록 만들었습니다.

그런데 새로운 문제가 생겼습니다.

동료가 "AI가 별로인 답을 몇 번 했어요"라고 이야기하지만 언제였는지는 기억하지 못합니다. 수십 개의 대화를 하나씩 읽어야 한다면 시간이 많이 걸립니다.

이번에는 사용자가 답변을 본 자리에서 좋아요, 별로예요, 신고를 누르면 해당 트레이스에 점수가 자동으로 기록되도록 만듭니다.

2. 중학생도 이해하는 핵심 개념

① Score란 무엇인가?

Score는 AI가 만든 결과물에 사용자가 평가를 붙이는 기능입니다.

예를 들어 온라인 쇼핑몰에서 상품을 구매한 뒤 별점을 주는 것과 비슷합니다.

사용자 반응	Langfuse 점수 이름	값
좋아요	user-feedback	1
별로예요	user-feedback	0
신고	report	1

여기서 좋아요와 별로예요는 `BOOLEAN` 방식으로 기록합니다.

- 좋아요 = `true`
- 별로예요 = `false`
- 신고 = `true`

신고 사유는 점수의 `comment`에 기록합니다.

이렇게 하면 나중에 Langfuse에서 별로예요가 붙은 트레이스만 골라낼 수 있습니다.

② 화면의 메시지와 Langfuse 트레이스를 연결해야 합니다

이번 챕터에서 가장 중요한 기술적 문제입니다.

사용자가 화면에서 보는 것은 답변 메시지이고, Langfuse에서 점수를 붙이는 대상은 트레이스입니다.

그런데 둘은 서로 다른 ID를 사용합니다.

화면과 Langfuse를 연결하는 과정

사용자 화면

답변 메시지 ID: `message_123`

사용자가 별로예요 버튼 클릭

메시지 ID로 트레이스 ID 찾기

Langfuse

Trace ID: `trace_456`

user-feedback = false

두 ID를 연결하는 방법은 두 가지입니다.

방법	설명	특징
메시지 meta에 Trace ID 저장	답변을 저장할 때 트레이스 ID도 함께 저장	나중에 다시 열어도 연결 가능
done 이벤트에 Trace ID 포함	스트리밍이 끝날 때 화면으로 트레이스 ID 전달	기존 저장 구조를 변경하지 않아도 됨

어느 방법이든 핵심은 사용자가 반응한 메시지가 어떤 트레이스에서 만들어졌는지 정확하게 찾아내는 것입니다.

3. 실제 구현 시 주의할 점

① 화면과 서버의 역할

사용자 화면에는 다음 세 가지 버튼을 둡니다.

AI 답변 말풍선

오늘은 서울 날씨가 맑을 것으로 예상됩니다.

좋아요

별로예요

신고

강의의 기능 구성을 이해하기 위한 예시 화면입니다.

각 버튼을 누르면 서버가 반응을 받아 Langfuse에 점수를 기록합니다.

신고 버튼은 기존 기능을 유지합니다.

② 반드시 서버에서 사용자 권한을 확인해야 합니다

예를 들어 A 사용자가 B 사용자의 답변을 보고 임의로 별로예요 점수를 남긴다면 평가 데이터가 오염됩니다.

따라서 서버가 다음을 확인해야 합니다.

1. 로그인한 사용자가 누구인지 확인합니다.
2. 반응 대상 메시지가 누구의 것인지 확인합니다.
3. 두 사용자가 일치하는지 검사합니다.
4. 일치할 때만 Langfuse에 점수를 기록합니다.

이것은 단순한 기능 구현이 아니라 서비스의 데이터 신뢰성을 지키는 보안 절차입니다.

③ Langfuse 키가 없을 때도 서비스는 정상 작동해야 합니다

Langfuse 키가 없는 환경에서는 점수 전송만 건너뛰고, 사용자의 반응 API는 오류 없이 처리하도록 설계합니다.

즉, 모니터링 도구가 잠시 작동하지 않더라도 AI 서비스 자체는 정상적으로 운영되어야 합니다.

4. 직접 실습: 별로인 답변만 찾아보기

이번 실습의 목적은 사용자의 평가가 실제 Langfuse 트레이스에 연결되는지 확인하는 것입니다.

실습 순서

1. 로컬에서 AI 친구에게 질문합니다.
2. 답변에 좋아요, 별로예요, 신고를 각각 눌러 봅니다.
3. Langfuse의 `Tracing > Traces`에서 해당 트레이스를 엽니다.
4. `user-feedback`, `report` 점수가 붙었는지 확인합니다.
5. 검색창에 다음 조건을 입력합니다.

```
scores."user-feedback":false
```

이 조건은 별로예요가 붙은 트레이스만 찾아줍니다.

여기에 User 필터까지 적용하면 특정 사용자가 별로라고 평가한 답변만 볼 수 있습니다.

Langfuse 검색 조건 예시

`scores."user-feedback":false`

검색 결과

별로예요 평가가 붙은 트레이스만 표시

BOOLEAN 점수는 숫자 0·1이 아니라 false·true로 검색합니다.

이제 해당 트레이스를 열어 챕터 05에서 배운 순서대로 분석하면 됩니다.

- choose-tools: 도구 선택이 잘못되었는가?
- tool execution: 외부 도구가 잘못된 결과를 반환했는가?
- reply: AI가 잘못된 답변을 만들었는가?
- memory-update: 기억 정리에 문제가 있었는가?

이렇게 하면 사용자의 불만을 실제 개선 작업으로 연결할 수 있습니다.

5. 사용자 피드백을 해석할 때 주의할 점

이번 챕터에서 특히 중요한 운영 관점입니다.

예를 들어 전체 답변 중 별로예요가 붙은 트레이스가 3%라고 가정해 보겠습니다.

이것을 곧바로 전체 답변의 97%가 만족스럽다는 뜻으로 해석하면 안 됩니다.

이유는 다음과 같습니다.

- 모든 사용자가 버튼을 누르지는 않습니다.
- 만족한 사용자는 반응을 남기지 않을 가능성이 큽니다.
- 불만이 큰 사용자는 상대적으로 버튼을 더 자주 누를 수 있습니다.
- 따라서 피드백을 남긴 사람의 반응이 전체 사용자 의견을 대표하지 않습니다.

그래서 피드백 점수는 문제가 있을 가능성이 높은 답변을 먼저 읽기 위한 우선순위 자료로 활용해야 합니다.

전체 품질을 판단하려면 다음을 함께 봐야 합니다.

평가 방법	역할
사용자 피드백	실제 사용자의 반응
사람이 직접 읽고 평가	답변의 정확성과 적절성 검증
자동 채점	대량의 답변을 일관된 기준으로 평가
Langfuse Score	평가 결과를 트레이스와 연결하고 검색

또한 신고 점수는 부적절한 AI 생성 콘텐츠를 신고하고 대응한 기록으로도 활용할 수 있습니다.

6. 실제 프로젝트에 적용한다면

이번 기능은 사용자가 직접 평가하는 AI 서비스라면 대부분 적용할 수 있습니다.

프로젝트	피드백 활용 방법
AI 친구 에이전트	공감이 부족하거나 틀린 답변 수집
MSM	사용자의 원래 문체를 잘 보존했는지 평가
ASP 컨설팅 에이전트	사업 분석과 제안의 적절성 평가
뉴스레터 에이전트	뉴스 선정과 요약 품질 평가
Travel Creator	여행 일정과 추천 장소의 만족도 평가

특히 MSM에서는 사용자가 작성한 원래 글의 느낌을 AI가 얼마나 잘 살렸는지 평가받을 수 있습니다.

예를 들어 다음과 같은 평가 항목으로 발전시킬 수 있습니다.

- 원문 문체를 잘 보존했는가?
- 원래 이야기의 의미가 바뀌지 않았는가?
- 이야기가 자연스럽게 구성되었는가?
- 사용자가 원하는 방향으로 수정되었는가?

이런 피드백이 쌓이면 단순히 AI가 글을 생성하는 서비스를 넘어, 사용자 만족도를 지속적으로 개선하는 스토리 제작 서비스로 발전시킬 수 있습니다.

7. AI 창업자가 반드시 알아야 하는 이유

이번 챕터는 AI 서비스의 품질 개선 과정에서 사용자 참여가 얼마나 중요한지를 보여줍니다.

기존에는 개발자가 AI 답변을 확인하고 문제가 있는지 판단했습니다.

하지만 실제 고객이 사용하는 서비스에서는 개발자가 모든 대화를 읽을 수 없습니다.

따라서 사용자가 불만을 표시한 순간을 기록하고, 그 기록을 실제 AI 실행 과정과 연결해야 합니다.

이를 통해 다음과 같은 개선 순환 구조를 만들 수 있습니다.

① AI 답변 생성

② 사용자 평가

좋아요 · 별로예요 · 신고

③ Langfuse에 Score 기록

④ 문제 트레이스 분석

⑤ 프롬프트·모델·도구 개선

개선된 AI가 다시 사용자에게 답변하는 지속적인 품질 개선 구조입니다.

이것이 AI 서비스를 실제 사업으로 운영할 때 필요한 피드백 기반 개선 체계입니다.

8. 객관식 퀴즈 정답 및 해설

정답 2번

버튼을 누르는 사용자는 일부이고 불만이 있을 때 더 누르므로, 3%는 읽을 순서를 정하는 단서이고 전체 품질은 직접 읽고 나눈 결과와 함께 봐야 합니다.

이유는 사용자 피드백이 전체 사용자의 만족도를 그대로 대표하지 않기 때문입니다.

- 1번: 반응하지 않은 97%가 모두 만족했다고 볼 수 없습니다.
- 3번: 3%라는 숫자만으로 문제를 무시할 수 없습니다.
- 4번: 점수가 정상적으로 붙었다면 계측 오류라고 볼 근거가 없습니다.

9. 오늘 반드시 기억할 내용

1. Langfuse Score는 사용자 평가를 AI 트레이스에 연결하는 기능입니다.
2. 좋아요는 `user-feedback=true`, 별로예요는 `user-feedback=false`로 기록합니다.
3. 신고는 `report=true`로 기록하고 신고 사유는 comment에 남깁니다.
4. 화면의 message_id와 Langfuse의 trace_id를 연결해야 합니다.
5. 서버에서 로그인 사용자와 메시지 소유자를 반드시 확인해야 합니다.
6. `scores."user-feedback":false`로 별로예요가 붙은 트레이스를 검색할 수 있습니다.
7. 사용자 피드백은 전체 품질의 절대적인 지표가 아니라 문제 분석의 우선순위 자료입니다.
8. 사람의 직접 평가와 자동 채점을 함께 활용해야 합니다.

10. 한 문장으로 정리

사용자의 좋아요·별로예요·신고를 Langfuse Score로 연결하면 문제가 있었던 AI 답변을 쉽게 찾아내고, 실제 사용자 피드백을 바탕으로 AI 서비스 품질을 지속적으로 개선할 수 있습니다.

지금까지의 학습 흐름을 연결하면: 챕터 04에서 AI 내부 동작을 기록하고, 챕터 05에서 계측 코드를 적용하고, 챕터 06에서 속도와 비용을 분석하고, 챕터 07에서 사용자와 버전을 구분하고, 챕터 08에서 사용자 평가까지 수집하게 되었습니다.

이제 AI 에이전트의 내부 동작뿐 아니라 실제 사용자의 반응까지 데이터로 확인할 수 있는 운영 기반을 갖추게 된 것입니다.

Chapter09

/강의정리 챕터 09. 동료들의 말에서 기능으로

1. 학습 목적

이번 챕터는 지금까지 배운 Langfuse와 LLMOps 내용을 종합하고 마무리하는 시간입니다.

핵심은 사용자의 불만을 실제로 확인할 수 있는 데이터와 기능으로 바꾸는 것입니다.

처음에는 동료들이 다음과 같이 이야기했습니다.

- AI가 엉뚱한 답을 해요.
- 가끔 답변이 너무 느려요.
- 메시지 수보다 요금이 많이 나와요.
- 전에 이야기한 내용을 기억하지 못해요.
- 업데이트한 뒤 느려진 것 같아요.
- 별로인 답이 몇 번 있었어요.

이런 문제는 단순한 서버 로그만으로는 원인을 찾기 어렵습니다.

이번 강의까지 진행하면서 각각의 불만을 Langfuse의 구체적인 기능과 연결했습니다.

2. 동료들의 불만을 해결하는 Langfuse 기능

동료의 불만	확인할 Langfuse 화면	확인할 내용
엉뚱한 날씨 답	Trace 상세 화면	도구 결과와 답변 생성 과정에서 처음 어긋난 단계
가끔 느린 답	Latency 정렬 및 시간 막대	단계별 지연 시간, 첫 글자까지 걸린 시간
메시지보다 많은 요금	Generation, Dashboards	모델 호출 횟수, 토큰, 단계별 비용
이사 이야기를 잊음	Users, Sessions	기억 저장, 전달, 활용 과정
업데이트 뒤 느려짐	Environment, Version 필터	배포 버전별 지연 시간 비교
별로인 답이 있었음	Score로 검색	좋아요·별로예요·신고가 붙은 트레이스

여기서 중요한 것은 문제가 발생한 뒤 어디를 확인해야 하는지 미리 정해 놓았다는 점입니다.

문제가 발생할 때마다 무작정 로그를 뒤지는 것이 아니라, 문제 유형에 맞는 화면과 데이터를 확인하면 됩니다.

3. Langfuse의 전체 동작 구조

이번 강의에서 배운 Langfuse의 작동 방식도 정리해 보겠습니다.

Langfuse 데이터 수집 및 저장 과정

AI 에이전트

Trace / Span / Generation 생성

Langfuse SDK

데이터를 버퍼에 모음

백그라운드 전송

모아 둔 데이터를 서버로 전송

Langfuse 서버

대기열 → 워커 → 데이터 저장

Langfuse 화면

Tracing / Users / Sessions / Scores / Dashboards

SDK의 버퍼와 flush()

SDK는 관찰 데이터를 매번 즉시 전송하지 않고 일정량 모아서 백그라운드에서 보냅니다.

이렇게 하면 사용자에게 답변을 보내는 과정이 계측 데이터 전송 때문에 매번 늦어지는 것을 줄일 수 있습니다.

다만 몇 가지 주의점이 있습니다.

- Langfuse 화면에 데이터가 몇 초 늦게 나타날 수 있습니다.
- 짧은 스크립트는 종료되기 전에 전송되지 않은 데이터가 남을 수 있습니다.
- 따라서 프로그램 종료 전에 `flush()`를 호출해 남은 데이터를 전송해야 합니다.
- Cloud Run에서는 요청 종료 후 CPU 사용 조건을 고려해 데이터 전송이 누락되지 않도록 해야 합니다.

Cloud와 셀프 호스팅의 차이

둘의 가장 큰 차이는 서버를 누가 운영하느냐입니다.

구분	Langfuse Cloud	Self-hosting
서버 운영	Langfuse 측	우리 조직
설치 및 관리	상대적으로 간편	직접 설치하고 관리
인프라 운영	제공업체가 담당	우리 조직이 담당
데이터 관리	서비스 제공 정책에 따름	직접 통제할 범위가 넓음

초기 1인 창업이나 MVP 단계에서는 Cloud를 활용하면 인프라 관리 부담을 줄일 수 있습니다. 고객 데이터의 민감도와 보안 요구가 커지면 셀프 호스팅도 검토할 수 있습니다.

4. 구조를 먼저 정하고 계측하는 것이 중요합니다

이번 강의 전체에서 제가 가장 중요하게 생각하는 부분입니다.

계측은 단순히 코딩 에이전트에게 "Langfuse를 연결해 줘"라고 요청하는 작업이 아닙니다.

먼저 서비스에서 어떤 문제를 확인하고 싶은지 정해야 합니다.

예를 들어 AI 친구 에이전트에서 기억 실패를 찾고 싶다면 다음을 결정해야 합니다.

1. 어떤 단계를 기록할 것인가?
2. 각 단계에서 무엇을 입력하고 출력하는가?
3. 사용자를 어떤 기준으로 구분할 것인가?
4. 대화 세션은 어디서 시작하고 끝나는가?
5. 개인정보는 어디까지 기록할 것인가?
6. 사용자가 남긴 평가를 어떻게 연결할 것인가?

그다음 코딩 에이전트에게 공식 문서 근거와 함께 구현을 요청해야 합니다.

즉, 무엇을 관찰할지 결정하는 것은 서비스 기획자의 역할이고, 이를 코드로 구현하는 것은 개발자 또는 코딩 에이전트의 역할입니다.

특히 비개발자라도 서비스의 업무 흐름과 고객 문제를 이해하고 있다면 계측 구조를 설계할 수 있다는 점이 중요합니다.

5. 실제 프로젝트에 적용한다면

이번 학습은 앞으로 만들 AI 서비스의 공통 운영 기반으로 활용할 수 있습니다.

프로젝트	관측해야 할 핵심 내용
AI 친구 에이전트	대화 품질, 기억 저장·활용, 응답 속도, 비용

프로젝트	관측해야 할 핵심 내용
MSM	원문 분석, 문체 보존, 스토리 생성 품질, 사용자 평가
ASP 컨설팅 에이전트	자료 분석 정확도, 고객별 이력, 보고서 품질, 고객 만족도
뉴스레터 에이전트	뉴스 수집, 기사 평가, 요약 품질, 발송 성공률
Travel Creator	장소 검색 정확도, 일정 생성 품질, 외부 API 지연

특히 여러 고객의 업무를 처리하게 될 ASP 컨설팅 에이전트에서는 `user_id`, `session_id`, `environment`, `version`이 중요합니다.

고객별 분석 기록을 구분하고, 프롬프트나 모델을 변경한 뒤 결과가 좋아졌는지 확인할 수 있기 때문입니다.

6. 아직 해결되지 않은 문제와 한계

이번 강의에서 반드시 기억해야 할 부분입니다.

Langfuse를 연결했다고 AI 품질 문제가 모두 해결되는 것은 아닙니다.

아직 남아 있는 문제	이유	앞으로 필요한 작업
답변의 실제 품질 판단	트레이스는 과정만 보여줌	사람이 직접 평가하고 자동 채점
피드백의 대표성	일부 사용자만 버튼을 누름	다양한 평가 데이터 확보
대량 트레이스 분석	수백 개를 하나씩 읽기 어려움	평가 자동화 및 분석 체계
비용과 지연의 정확성	초기 테스트 데이터가 적음	실제 사용 데이터 축적
개인정보 보호	대화 원문이 외부 서비스에 전송될 수 있음	마스킹, 접근 권한, 보관 기간, 사용자 안내

여기서 가장 중요한 한계는 관측과 평가가 서로 다르다는 점입니다.

- 관측(Observability): AI가 무슨 일을 했는지 보여줍니다.
- 평가(Evaluation): AI가 한 일이 좋았는지 나빴는지 판단합니다.

Langfuse에 기록된 트레이스만으로 답변의 품질을 자동으로 확정할 수는 없습니다.

또한 개인정보를 기록하는 경우에는 서비스를 공개하기 전에 어떤 정보를 수집하고, 무엇을 가리며, 얼마나 보관할지 반드시 점검해야 합니다.

7. 주관식 퀴즈 답안 예시

강의에서 처음 고른 동료의 말이 무엇인지에 따라 답변을 작성하면 됩니다. 아래는 '이사 이야기를 기억하지 못한다'는 문제를 선택했을 때의 예시입니다.

처음에 동료가 "다음 주에 이사 간다고 말했는데 AI가 기억하지 못한다"고 이야기했습니다.

오늘은 Langfuse의 Users와 Sessions 화면에서 해당 사용자의 대화 기록을 찾아 확인할 수 있게 되었습니다. 먼저 이사 이야기를 했던 트레이스의 memory-update 출력을 확인하고, 이후 질문을 한 트레이스의 reply 입력에 해당 기억이 포함되어 있는지 살펴보면 기억이 저장되지 않은 것인지, 저장은 되었지만 답변에 전달되지 않은 것인지 구분할 수 있습니다.

다만 현재의 관측만으로 AI가 실제로 기억을 잘 활용했는지, 장기간 대화에서도 일관되게 기억하는지 완전히 판단하기는 어렵습니다. 실제 사용자의 대화가 더 쌓여야 하며, 사람이 직접 답변을 평가하고 자동 채점도 함께 적용해야 합니다. 개인정보 마스킹과 보관 기간도 실제 서비스 공개 전에 추가로 점검해야 합니다.

8. 오늘 반드시 기억할 내용

1. 사용자 불만을 해결하려면 먼저 문제에 맞는 관측 화면과 기록 항목을 정해야 합니다.
2. Langfuse SDK는 관찰 데이터를 버퍼에 모아 백그라운드로 전송합니다.
3. 짧은 프로그램이나 요청 종료 시에는 `flush()`가 필요할 수 있습니다.
4. Cloud와 Self-hosting의 차이는 서버 운영 주체입니다.
5. 계측 구조와 기록할 정보는 서비스의 업무 흐름을 이해하는 사람이 먼저 설계해야 합니다.
6. 코딩 에이전트에는 설계 내용, 공식 문서 근거, 보안 조건, 테스트 조건을 함께 전달해야 합니다.
7. 관측은 무슨 일이 일어났는지 보여주지만, 품질 평가는 별도의 과정이 필요합니다.
8. 실제 서비스에서는 개인정보 보호와 데이터 보관 정책까지 고려해야 합니다.

9. 한 문장으로 정리

이번 강의의 핵심은 동료들의 막연한 불만을 Langfuse의 관측 기능으로 연결해, AI 에이전트의 문제를 데이터로 발견하고 개선할 수 있는 운영 체계를 설계하는 것입니다.

10. 이번 Langfuse 학습 전체를 돌아보며

챕터 01~09 학습 마무리

이번 과정은 단순히 Langfuse라는 도구의 사용법을 배운 것이 아닙니다. AI 서비스를 실제 고객에게 제공하기 위해 필요한 운영 체계를 단계적으로 배운 것입니다.

학습 단계	핵심 역량
챕터 01~03	LLMOps와 Langfuse 환경 이해

학습 단계	핵심 역량
챕터 04~05	관측 구조 설계와 코드 계측
챕터 06	속도와 비용 분석
챕터 07	사용자·세션·배포 버전 구분
챕터 08	사용자 피드백 수집
챕터 09	전체 운영 체계의 연결과 한계 이해

제가 생각하는 가장 중요한 학습 성과는 AI 서비스를 만드는 사람에서 AI 서비스를 운영하고 개선하는 사람으로 관점이 확장되었다는 것입니다.

앞으로 MSM이나 ASP를 실제 서비스로 출시할 때도 처음부터 모든 기능을 완벽하게 만들기보다는, 고객의 문제를 관측하고 원인을 찾아 개선할 수 있는 구조를 먼저 갖추는 것이 중요합니다.